System Design Encyclopedia

A historical, causal guide to system design: concepts, trade-offs, examples and diagrams.

Parts 1-13: reference tables. Parts 14-19: deep dives with examples and diagrams.

Part 1 – Computing Foundations (A) and Single-Machine Software (B)

1.1 Data representation

Concept	Definition	Pressure / why it matters	Design impact
Bits & bytes	Bit = 0/1; byte = 8 bits, smallest addressable unit	Hardware is binary; need uniform units	Capacity math (KB/MB/GB, 1024 vs 1000), endianness in wire formats
Encoding	Mapping text/data to bytes (ASCII→Unicode/UTF-8)	Multi-language systems broke on 7-bit ASCII	Always declare charset; UTF-8 default; byte length ≠ char length
Serialization	Object → bytes (JSON, XML, Protobuf, Avro, MessagePack, Java serialization)	Need to store/send structures	JSON: readable, big, slow; Protobuf/Avro: compact, schema-evolution rules; Java native serialization: insecure/fragile
Compression	Reduce size (gzip, zstd, snappy, lz4)	Bandwidth/storage cost	CPU↔size trade-off; snappy/lz4 for speed, zstd/gzip for ratio; compressed data resists range-reads
Hashing	Fixed-size digest (MD5/SHA-2, murmur, xxHash)	Fast lookup, integrity, partitioning	Hash tables, sharding, dedup; crypto vs non-crypto hashes differ; collisions
Checksums	CRC/Adler to detect corruption	Disks/networks flip bits	Used in TCP, filesystems (ZFS), storage engines, Kafka records
Encryption basics	Symmetric (AES) vs asymmetric (RSA/ECC), key exchange	Confidentiality over shared networks	Key management is the real problem; CPU cost (AES-NI) small

1.2 CPU, memory, storage hierarchy

```
Registers (~0.3 ns) -> L1 (~1 ns) -> L2 (~4 ns) -> L3 (~15 ns) -> RAM (~100 ns)
 -> NVMe SSD (~100 us) -> SATA SSD (~200 us) -> HDD (~5-10 ms)
 -> Same-DC network RTT (~0.5 ms) -> Cross-region RTT (~50-150 ms)
```

Concept	Definition	Design consequence
CPU & instruction cycle	Fetch→decode→execute→write-back, pipelined, superscalar	Branch mispredictions & cache misses dominate; clock speed plateaued ~2005 → multicore → concurrency became mandatory
Registers	Fastest storage inside CPU	Compilers manage; atomics operate here/cache lines
CPU caches	L1/L2/L3 SRAM, cache lines (64 B)	Locality matters; false sharing between threads; data-oriented layouts
Locality	Temporal & spatial locality	Basis of every cache in the stack (CPU, OS, DB buffer pool, CDN)
RAM	Volatile, byte-addressable	100–1000× faster than disk; in-memory DBs (Redis) are a reaction to cheap RAM
Virtual memory	Per-process address space, paging, TLB	Isolation; swapping kills latency; mmap-based DBs; OOM killer
NUMA	Memory attached to specific CPU sockets	Remote memory access slower; pin threads/memory for DBs and JVMs
HDD	Spinning platters; seek ~5–10 ms; ~150 MB/s sequential	Random I/O is ~100× slower than sequential → **B-trees w/ big pages, WAL (sequential), LSM trees**

Concept	Definition	Design consequence
SSD/NVMe	Flash; no seek; write amplification, erase blocks, wear	Random reads cheap; random small writes cause WA; LSM tuned for SSD; TRIM
Sequential vs random	Access pattern	Append-only logs (Kafka, WAL) exploit sequential speed
I/O, interrupts, DMA	Devices signal CPU via interrupts; DMA moves data w/o CPU	Zero-copy (sendfile), io_uring, epoll; kernel bypass (DPDK) for extreme throughput
Buses	PCIe, memory bus	Bandwidth ceilings for NICs/GPUs/NVMe
Time & clocks	Wall clock (NTP-synced, can jump) vs monotonic; drift	Never measure durations with wall clock; basis for clock-skew problems (Part 6)

1.3 Operating systems & concurrency

Evolution: batch (1950s, one job at a time, CPU idle during I/O) → multiprogramming (overlap I/O) → time-sharing (1960s, interactivity; Unix) → personal computers → multicore/preemptive multitasking.

Concept	Definition	Pressure → trade-off
Process	Isolated address space + resources	Isolation (fault, security) at cost of heavy creation & IPC
Thread	Execution unit sharing process memory	Cheaper, shared state → races, deadlocks
Context switch	Save/restore CPU state	Thousands of threads → switch overhead & cache thrash → **event loops / async I/O / green threads / virtual threads**
Concurrency vs parallelism	Concurrency = structure (interleaving); parallelism = simultaneous execution	Concurrency helps I/O-bound; parallelism helps CPU-bound
Scheduling	Round-robin, priority, CFS, work stealing	Fairness vs latency vs throughput; priority inversion
Synchronization	Mutex/lock, semaphore, monitor, condition variable, read-write lock, spinlock, barrier	Correctness vs contention; lock convoy
Atomic ops / CAS / memory model	Hardware-guaranteed indivisible ops; happens-before	Lock-free structures; ABA problem; memory ordering subtleties
Race condition	Outcome depends on timing	Needs locks, immutability, or confinement
Deadlock	Cyclic wait (4 Coffman conditions)	Lock ordering, timeouts, try-lock
Starvation / livelock	Thread never gets resource / keeps retrying	Fair locks, backoff
IPC	Pipes, sockets, shared memory, message queues, signals	Shared memory fastest but unsafe; sockets generalize to network → **distribution**
Models beyond threads	Actor model, CSP (channels), STM, async/await, reactor/proactor	*Parallel evolution* answers to "shared mutable state is hard"

Java connection (this repo): `threads/` package demos — deadlock, latch/barrier, semaphore, ForkJoin, CompletableFuture, locks, wait/notify.

1.4 Files, filesystems, journaling, virtualization

Concept	Definition	Pressure → trade-off
File/filesystem	Named byte streams; inode, directory, block allocation (ext4, NTFS, XFS, ZFS)	Persistence abstraction; fragmentation

Concept	Definition	Pressure → trade-off
Journaling / WAL	Log intent before applying (ext4, NTFS)	Crash consistency after power loss; write amplification. **Same idea reappears as DB WAL, Raft log, Kafka**
Copy-on-write FS	ZFS/Btrfs, snapshots	Cheap snapshots/checksums; fragmentation
fsync & durability	Force data to stable storage	Durability vs latency; lying disks/controllers
Virtualization	Hypervisor (Type 1/2) emulates hardware; VMs	Server consolidation, isolation (reaction to cost: 10% CPU utilization of bare metal) → overhead, boot time, image size → **containers** (Part 8)

1.5 Single-machine software (B)

- **Single-process app:** all logic, state, storage in one process. Simple, fast, no network. Limits: one machine's CPU/RAM/disk; one crash = total outage; single user.
- **Batch processing:** run job over bounded input (payroll). High throughput, no interactivity → *interactive/time-sharing*.
- **Mainframe & time-sharing:** centralized compute, dumb terminals. Strong reliability, expensive, vendor lock-in → *PC / client-server* (reaction to cost).
- **Desktop apps:** local compute, local files. Data siloed, no sharing → *client-server* (reaction to business requirements: shared data).

★ Why foundations shape design (summary)

1. Disk vs RAM vs network latency ratios decide *where* data lives (cache, replicate, shard).
2. Sequential I/O superiority → logs, LSM, append-only everything.
3. Multicore → concurrency control → every shared-state bug class.
4. Clocks unreliable → logical time, fencing tokens (Part 6).
5. Crash consistency (journaling) → WAL/consensus logs across the stack.

Part 2 – Networking

Pressure: once software left a single machine, **every call became slow, lossy, reorderable and attackable.** Networking concepts define what distributed systems can assume.

2.1 Foundations

Concept	Definition	Architectural impact
Topologies	Bus, star, ring, mesh, tree	Star (switched) dominates LAN; mesh in data-center fabrics (Clos/leaf-spine)
LAN / WAN	Local vs wide-area	WAN: latency, loss, cost → CDNs, geo-replication
Circuit switching	Dedicated path (telephony)	Predictable but wasteful → **packet switching**
Packet switching	Data split into independently routed packets (ARPANET 1969)	Resilient, shared; reordering, loss, jitter → transport protocols
OSI (7) / TCP-IP (4)	L1 physical, L2 link, L3 network, L4 transport, L5-7 app	"L4 vs L7" load balancing; debugging by layer
Ethernet / switching	Frames by MAC; switches learn MAC tables	Broadcast domains, VLANs
ARP	IP→MAC resolution	ARP spoofing; L2 limits
IPv4 / IPv6	32-bit vs 128-bit addresses	IPv4 exhaustion → NAT, CGNAT → IPv6 (reaction to scale)
Subnetting/CIDR	Address partitioning	VPC design, security zones, avoid overlapping CIDRs when peering
Routing / BGP	Interior (OSPF) & inter-domain (BGP) path selection	Internet outages from BGP leaks; Anycast relies on it
DHCP	Dynamic IP assignment	Ephemeral IPs → service discovery needed
NAT	Many private IPs → one public	Breaks end-to-end, hurts P2P (STUN/TURN), port exhaustion
DNS	Hierarchical name→IP; TTL caching	Cheapest global load-balancing/failover; TTL staleness; DNS is a classic SPOF/outage

2.2 Transport

Protocol	Properties	Trade-off	Successor/alternative
TCP	Reliable, ordered, connection-oriented, 3-way handshake, flow control (window), congestion control (slow start, CUBIC, BBR)	Handshake RTT, head-of-line blocking, slow start hurts short flows	QUIC (partial successor)
UDP	Unreliable datagrams	No ordering/retransmit; app must handle	Used for DNS, VoIP, games, QUIC
QUIC	UDP-based, TLS 1.3 built-in, multiplexed streams, 0/1-RTT, connection migration	User-space CPU cost, middlebox blocking	Basis of HTTP/3
Sockets/ports	OS endpoint abstraction (IP:port)	Ephemeral port exhaustion, TIME_WAIT	Connection pooling
Flow control	Receiver limits sender (window)	Protects receiver	App-level **backpressure** mirrors it

Protocol	Properties	Trade-off	Successor/alternative
Congestion control	Sender infers network capacity	Fairness vs throughput	BBR for lossy/long links
Keep-alive / pooling	Reuse connections	Saves handshakes; stale connections, LB idle timeouts	HTTP/2 multiplexing
Timeouts / retries	Bound wait; reattempt	Retry storms, duplicates → idempotency, backoff+jitter (Part 6)	

2.3 Application protocols

Version	Key change	Problem solved	New problem
HTTP/1.0	One request per connection	—	Handshake per request
HTTP/1.1	Persistent connections, pipelining, Host header	Cost of connections	App-level head-of-line blocking; 6 conns/host hacks (sharding, sprites)
HTTP/2	Binary, multiplexed streams, HPACK, server push	HOL at HTTP layer	TCP-level HOL on loss; push rarely useful
HTTP/3	HTTP over QUIC	TCP HOL, faster handshake, mobility	UDP blocking, CPU cost, observability tooling
SSL → TLS	SSL 2/3 (1990s, broken) → TLS 1.0–1.3	Confidentiality, integrity, server auth	Cert management/expiry, handshake latency (TLS 1.3 1-RTT, resumption)

2.4 Intermediaries

Component	What	Notes
Forward proxy	Client-side egress proxy	Filtering, caching, anonymity
Reverse proxy	Server-side front (Nginx, Envoy, HAProxy)	TLS termination, caching, compression, routing
L4 LB	Balances TCP/UDP by IP:port; very fast; connection-level	No content awareness; sticky by hash
L7 LB	Understands HTTP; routing by path/header; retries; auth	More CPU; can become bottleneck
LB algorithms	Round-robin, weighted, least-connections, least-latency, power-of-two-choices, consistent hash, IP hash	Stateful/sticky vs stateless
CDN	Geo-distributed caches near users	Latency/bandwidth/DDoS absorption; invalidation, cost
Anycast	Same IP announced from many sites via BGP	Nearest-site routing, DDoS dilution; flaps break TCP flows
Firewall / WAF	Packet filter / L7 attack filter	Perimeter defence; false positives; not enough alone (zero trust)
VPN	Encrypted tunnel (IPsec, WireGuard)	Remote access, site-to-site; perimeter trust model weaknesses
Service discovery	DNS, registry (Consul, etcd, Eureka), K8s Services	Needed because IPs are ephemeral

2.5 Network realities

Concept	Meaning	Design response
Latency	Delay (propagation ≥ distance/ ~200,000 km/s in fiber)	Geo placement, caching, fewer round trips, batching
Bandwidth	Throughput capacity	Compression, CDN, bulk transfer paths; bandwidth-delay product
Jitter	Latency variance	Buffers (media), tail-latency mitigation (hedging)
Packet loss	Drops	Retransmit, FEC, idempotent ops
Head-of-line blocking	One stalled item blocks others behind it	HTTP/2 → QUIC; per-partition queues
Partition	Segment unreachable	CAP decisions (Part 4/6)
Backpressure	Slow consumer signals producer	Bounded queues, reactive streams, credits
Network security	Segmentation, ACLs, TLS/mTLS, DDoS protection, zero trust	See Part 7

2.6 Fallacies of distributed computing (Deutsch/Gosling)

The network is reliable · latency is zero · bandwidth is infinite · the network is secure · topology doesn't change · there is one administrator · transport cost is zero · the network is homogeneous.

★ Example – What happens on `GET https://shop.com/`

```
Client -> DNS (cache -> resolver -> authoritative; maybe GeoDNS)
       -> TCP/QUIC handshake -> TLS handshake (SNI)
       -> Anycast edge / CDN (cache hit? return)
       -> L4 LB -> L7 LB / API gateway (auth, rate limit)
       -> Service -> cache -> DB
       <- response (compressed, cacheable headers)
```

Each hop adds latency, a failure mode, a timeout, and a security boundary.

Part 3 – Application Architectures and API Styles

3.1 Evolution chain

```
Single process -> Mainframe/terminal -> 2-tier client-server (fat client + DB)
 -> 3-tier web (browser / app server / DB) -> N-tier
 -> Layered / MVC family -> Component/plugin -> Modular monolith
 -> SOA (ESB) -> Microservices -> Serverless (FaaS/BaaS)
 parallel: Event-driven, CQRS/ES, Actors, P2P, Space-based, Pipeline
```

3.2 Tiered architectures (C–D)

Architecture	Emerged	Replaced/supplemented	Pros	Cons	Next
Client-server	1980s	Mainframe/desktop silos	Shared data, central control	Server SPOF, LAN assumptions	n-tier
Thick client (2-tier)	Late 80s–90s (VB/PowerBuilder + DB)	Terminals	Rich UI, fast dev	Business logic on every client, DB connections per client, painful deployment	3-tier
Thin client / web app	Mid 90s	Fat client	Zero install, central deploy	Page reloads, server-heavy	AJAX/SPA, mobile
3-tier	90s	2-tier	Logic centralized, DB pooled, independently scalable tiers	App tier becomes a monolith	N-tier / SOA
N-tier	2000s	3-tier	Cache/queue/search tiers added	Latency per hop, ops complexity	Microservices

3.3 UI/code-structure patterns

Pattern	Idea	Trade-off	Relation
MVC (1979 Smalltalk)	Model/View/Controller	View↔model coupling variants	Web frameworks (Spring MVC, Rails)
MVP	Presenter mediates, passive view	More boilerplate, testable	Partial successor of MVC (desktop)
MVVM	Data-binding ViewModel	Binding magic, debugging	WPF/Angular/SwiftUI
Layered	Presentation/Service/Domain/Data	Layers become pass-throughs; DB-centric	Default enterprise style
Hexagonal / Ports & Adapters	Domain core, adapters for I/O	Indirection	Reaction to DB-centric layering
Onion / Clean	Dependencies point inward	Ceremony for CRUD apps	Specialization of hexagonal
Component / Plugin / Microkernel	Core + extensions (Eclipse, IDEs, OS kernels)	Versioning/API stability	Used for extensible products
Modular monolith	One deployable, strict module boundaries	Needs discipline (enforce w/ tooling)	Often the right step before microservices

3.4 Distributed architectures

Architecture	Emerged	Pressure	Pros	Cons	Alternative/successor
★ Monolith	Always	Simplicity	Simple deploy, local calls, ACID txns, easy debugging	Scale whole thing, coupling, long builds, risky deploys, tech lock	Modular monolith / SOA
★ SOA	~1998–2005 (SOAP/WSDL, ESB)	Integrate enterprise systems	Reuse, standard contracts	Heavy XML, central ESB bottleneck/"smart pipes", governance	Microservices (reaction to ESB weight)
★ Microservices	~2011+ (Netflix, Amazon)	Org scale, independent deploy, independent scale	Team autonomy, polyglot, fault isolation, scale by service	Distributed-system problems: latency, partial failure, data consistency, observability, versioning, cost, Conway's law	Service mesh, sagas, platform eng.; or back to modular monolith
Event-driven (EDA)	2000s+	Decoupling, async	Loose coupling, extensibility	Hard tracing, eventual consistency, ordering	Event sourcing, workflow engines
DDD	2003 (Evans)	Complex domains	Bounded contexts → service boundaries	Learning curve, over-modeling	Underpins microservices boundaries
CQRS	2010	Read/write needs differ	Independent scaling & models	Sync lag, complexity	Often with event sourcing (not required)
Event sourcing	~2005+	Audit, temporal queries	Full history, rebuildable views	Schema evolution of events, snapshots, rebuild time, GDPR deletion	Plain audit tables/CDC
Actor systems (Erlang 1986, Akka, Orleans)	Parallel branch	Concurrency without shared state	Isolation, supervision, location transparency	Debugging, mailbox overflow, ordering across actors	Threads+locks, CSP
Pipeline / Pipes & Filters	Unix 1970s	Composable processing	Reuse, parallelism	Latency, schema coupling	Stream processing
Space-based (tuple space/IMDG)	2000s	Extreme scalability of stateful web	No DB bottleneck	Complex data consistency	Distributed caches, Kafka+KV
Peer-to-peer	1999 Napster, BitTorrent, blockchain	No central cost/control	Scales with users, resilient	Discovery, security, NAT, consistency	Client-server + CDN
Broker architecture	CORBA 90s, MQ	Decouple clients from servers	Location transparency	Broker SPOF/overhead	Service discovery/mesh
Blackboard	AI 1980s	Non-deterministic problem solving	Collaborative knowledge sources	Control complexity	Rule engines, multi-agent LLM orchestration (modern echo)
Serverless: FaaS	2014 AWS Lambda	Idle cost, ops burden	Scale-to-zero, pay-per-use	Cold starts, limits, vendor lock-in, observability, statelessness	Containers-on-demand (Cloud Run/Fargate)
BaaS	2010s (Firebase)	Skip backend	Fast MVP	Lock-in, limited logic	Custom backend later
BFF	2015	One API doesn't suit all clients	Tailored payloads	N backends to maintain	GraphQL gateway

Architecture	Emerged	Pressure	Pros	Cons	Alternative/successor
API gateway	2010s	Cross-cutting concerns	Auth, rate limit, routing	Bottleneck/SPOF, logic creep	Service mesh for east-west

★ Monolith vs modular monolith vs microservices vs SOA

Aspect	Monolith	Modular monolith	SOA	Microservices
Deploy unit	1	1	Few big services	Many small
Data	1 shared DB	1 DB, schema per module	Shared / per-service	DB per service
Comms	In-process	In-process via interfaces	ESB/SOAP	HTTP/gRPC/events
Consistency	ACID	ACID	Mixed	Eventual/sagas
Org fit	<~20 devs	10–100	Enterprise integration	Many autonomous teams
Main risk	Big ball of mud	Boundary erosion	ESB bottleneck	Distributed monolith

3.5 Communication & API styles

Style	Emerged	Pressure	Pros	Cons	Use when
RPC (Sun RPC, CORBA, RMI)	80s–90s	Call remote like local	Simple model	Hides network failures (fallacy), tight coupling	→ REST/SOAP reaction
SOAP/WSDL	1998	Interoperable enterprise contracts	WS-* security/transactions, strong contracts	Verbose, complex	Legacy B2B, banking
REST (2000 Fielding)	2000s	Web-scale simple resources	Cacheable, uniform, stateless, tooling	Over/under-fetching, chatty, no standard schema (OpenAPI helps)	Public APIs
GraphQL (2015)	Facebook	Mobile clients needing flexible queries	One round trip, typed schema	Caching harder, N+1, query-cost attacks, authorization per field	Many clients, aggregated data
gRPC (2015)	Google	Efficient internal RPC	Protobuf, HTTP/2, streaming, codegen	Browser support (grpc-web), debuggability	Internal east-west
WebSockets	2011	Bidirectional realtime	Low overhead push	Stateful connections, scaling LBs, reconnect logic	Chat, trading
SSE	2009	Server→client stream over HTTP	Simple, auto-reconnect	One-way, connection limits	Feeds/notifications
Long polling	2000s	Push emulation	Works anywhere	Held connections, latency	Fallback
Webhooks	2000s	Server→server callbacks	Simple event push	Delivery guarantees, security (signing), retries	SaaS integration

REST vs RPC: resource-oriented, cache-friendly vs action-oriented, efficient. REST vs GraphQL: multiple endpoints/HTTP caching vs single endpoint/client-shaped queries.

3.6 ★ Template-style deep dive: Microservices

1. **Definition** – small, independently deployable services around business capabilities, each owning its data.

2. **Context** – born from monolith pain at Amazon/Netflix; enabled by cloud, containers, CI/CD.
3. **Earlier** – monolith/SOA with ESB.
4. **Problem solved** – deploy coupling, scaling granularity, team blocking.
5. **How** – per-service repo/pipeline/DB; sync (REST/gRPC) + async (events); gateway at edge; discovery; config; observability.
6. **Components** – services, gateway, registry, config server, message broker, tracing, CI/CD, secrets.
7. **Diagram**

```
Client -> CDN -> LB -> API Gateway -> [Order Svc]--DB1
                                   -> [User Svc] --DB2
                                   -> [Pay Svc]  --DB3
             Order --events--> Broker --> Inventory, Notification
```

8. **Use cases** – large orgs, uneven scaling, polyglot needs.
9. **Pros** – independent deploy/scale/failure domains, team autonomy.
10. **Cons** – network latency, partial failure, no cross-service ACID, versioning, test environments, ops/cost, cognitive load.
11. **Failure modes** – cascading failure, retry storms, distributed monolith (lockstep deploys), chatty calls, schema drift.
12. **Perf/scale** – scale per service; p99 = compounded tail latency across fan-out.
13. **Consistency** – eventual; sagas, outbox, idempotency.
14. **Security** – more attack surface; mTLS, zero trust, per-service authz, secrets.
15. **Ops** – needs tracing, centralized logs, SLOs, platform team.
16. **Cost** – infra overhead (sidecars, duplicated runtimes) + people.
17. **Why next** – operational burden → service mesh, platform engineering, serverless, **or** modular monolith; consistency pain → sagas/event sourcing.
18. **Modern relevance** – dominant at scale, but "microservices-first" for small teams is often premature.
19. **Alternatives** – modular monolith, SOA, serverless.
20. **Interview** – decompose e-commerce; handle distributed transactions; service communication.
21. **Example** – Order service publishes `OrderPlaced` via outbox; Inventory reserves; Payment charges; saga compensates on failure.
22. **Summary** – autonomy ↑, complexity ↑↑, consistency ↓.

Part 4 – Databases, Transactions, Consistency, Recovery

4.1 Evolution of data storage

```
Flat files -> Hierarchical (IMS, 1960s) -> Network/CODASYL -> Relational (Codd 1970, SQL, System
R/Oracle)
 -> OO databases (90s, niche) -> ORMs
 -> Web scale: Key-value (Dynamo), Wide-column (Bigtable/Cassandra), Document (Mongo), Graph (Neo4j)
 -> Search (Lucene/Elasticsearch), Time-series, In-memory (Redis)
 -> Analytics: Data warehouse -> Data lake -> Lakehouse
 -> NewSQL / Distributed SQL (Spanner, CockroachDB), HTAP
 -> Vector DBs (embeddings, AI)
```

Type	Pressure that created it	Strengths	Weaknesses	Next/alternative
Flat files	Simplest persistence	Trivial	No concurrency, queries, integrity	DBMS
Hierarchical	Navigational access for mainframes	Fast for known paths	Rigid, many-to-many awkward	Network → relational
Network (CODASYL)	Many-to-many	Flexible links	Programmer navigates pointers; schema change pain	Relational (declarative)
★ Relational	Data independence, declarative SQL	ACID, joins, ad-hoc queries, mature	Vertical scaling limits, impedance mismatch, rigid schema	NoSQL (scale), ORMs
Object-oriented	Impedance mismatch	Natural object storage	No standard, poor tooling	ORM + relational; document DBs
Key-value	Massive scale, simple access (Dynamo 2007)	Horizontal scale, low latency	No rich queries/joins	Document, wide-column
Document	Flexible schema, nested data	Schema agility, locality	Weak joins, duplicated data	Relational JSON support
Wide-column	Huge write throughput (Bigtable 2006)	Linear scale, LSM	Query-model design upfront, limited transactions	Distributed SQL
Graph	Relationship-heavy queries	Multi-hop traversals	Hard to shard	Relational recursive CTE for small cases
Time-series	Metrics/IoT	Compression, downsampling, retention	Narrow use	Columnar OLAP
Search index	Full-text/relevance	Inverted index, faceting	Not source of truth, eventual	Vector search hybrid
In-memory	Cheap RAM	µs latency	Cost, durability (AOF/RDB)	Tiered/persistent memory
NewSQL/Distributed SQL	Want SQL+ACID at scale (Spanner 2012)	Global consistency, SQL	Commit latency (consensus), cost, clock dependence	Cell-based sharded SQL

Type	Pressure that created it	Strengths	Weaknesses	Next/alternative
Vector DB	Semantic search for embeddings	ANN (HNSW, IVF, PQ) similarity	Approximate recall, memory heavy, filtering complexity	pgvector, hybrid search
Data warehouse	OLAP separate from OLTP	Columnar, fast aggregates	Schema-on-write, cost, ETL	Data lake
Data lake	Cheap raw storage of all data	Flexibility, any format	"Data swamp", no ACID	Lakehouse (Delta/Iceberg/Hudi)
Lakehouse	Warehouse features on lake	ACID tables on object storage	Engine maturity, small-file issues	—
HTAP	Analytics on live data	Fresh analytics	Resource contention	CDC → separate OLAP

OLTP vs OLAP: OLTP = many small row-oriented transactions; OLAP = few large scans, columnar, aggregates.

4.2 Modeling

- **ER modeling:** entities, relationships, cardinality.
- **Normalization (1NF–BCNF):** removes redundancy/update anomalies. **Denormalization:** duplicate for read speed, at the cost of write complexity/inconsistency.
- **Keys:** primary (identity; choose monotonic vs random: UUIDv4 fragments B-tree, UUIDv7/ULID/snowflake are time-ordered), foreign (integrity; costly across shards).
- **Stored procedures:** logic near data (fast, hard to version/test). **Materialized views:** precomputed query results (refresh cost/staleness).
- **Schema evolution & migrations:** expand-and-contract; backward/forward compatibility; online DDL tools.
- **Polyglot persistence:** right store per workload; cost is operational variety and cross-store consistency (use CDC/outbox).

4.3 Storage engines & indexes

Structure	Idea	Reads	Writes	Used by
B-tree / B+ tree	Balanced page tree, leaves linked	O(log n), range scans excellent	In-place updates, random I/O, page splits	Postgres, MySQL InnoDB, Oracle
Hash index	Hash→location	O(1) point lookups	Resizing	Memory DBs, Bitcask
LSM tree	Memtable → immutable SSTables → compaction	Read amplification (bloom filters, indexes mitigate)	Sequential, very high throughput; write amplification via compaction	Cassandra, RocksDB, LevelDB, HBase
Bloom filter	Probabilistic set membership (no false negatives)	Skip SSTables	—	LSM, caches, penetration defence
Column store	Store by column	Compress + scan fast	Slow single-row writes	Redshift, ClickHouse, Parquet
Inverted index	Term→postings	Full-text	Segment merges	Lucene/Elasticsearch
Secondary index	Extra access path	Faster queries	Write cost; global vs local in distributed DBs	all

4.4 Transactions & concurrency

- **ACID:** Atomicity, Consistency (app invariants), Isolation, Durability. **BASE:** Basically Available, Soft state, Eventually consistent — reaction to scale/availability.
- **Isolation levels:** Read Uncommitted → Read Committed → Repeatable Read → Snapshot → Serializable. Anomalies: dirty read, non-repeatable read, phantom, lost update, write skew.
- **Serializability** (transactions behave as serial order) vs **linearizability** (single-object real-time recency) — different guarantees; *strict serializability* = both.
- **2PL (pessimistic):** shared/exclusive locks until commit; deadlocks → detection/timeouts. **OCC (optimistic):** validate at commit; retries under contention. **MVCC:** versions per row, readers don't block writers (Postgres, Oracle, InnoDB). **SSI:** serializable snapshot isolation.
- **Deadlocks:** lock ordering, short txns, timeouts.

4.5 Durability & recovery

Concept	Role
WAL	Log change before page write; sequential; basis of recovery and replication
Redo/Undo log	Redo re-applies committed; undo rolls back uncommitted (ARIES)
Checkpoint	Bound recovery time
Crash recovery	Analysis → redo → undo
Backup (full/incremental/differential)	Protection from logical corruption/human error (replication does **not** protect from `DROP TABLE`)
PITR	Base backup + WAL archive → restore to timestamp
DR	Cross-region copy + runbook; defined by RPO/RTO
Archival / retention / expiration	Cost + compliance (TTL, lifecycle policies, legal hold)
Data lineage / governance	Track origin/transforms; ownership, quality, access, classification, catalog
CDC	Stream DB changes from the log (Debezium) → search, cache, analytics, outbox

4.6 Scaling databases

Order of escalation (cheapest first): **index/query tuning → cache → vertical scale → read replicas → partition tables → shard → distributed SQL/NoSQL.**

Technique	Detail	Trade-off
Replication: leader-follower	Writes to leader, async/sync to followers	Replica lag, failover data loss (async), read-your-writes issues
Multi-leader	Writes in multiple regions	Conflict resolution (LWW, CRDTs, app merge)
Leaderless (Dynamo)	Quorum R+W>N, read repair, hinted handoff, anti-entropy	Tunable consistency, sloppy quorums, conflicting versions
Sync vs async replication	Durability vs latency/availability	Semi-sync compromise
Read replicas / write replicas	Read scale; write scale needs sharding	Staleness
Partitioning	Splitting data (vertical/horizontal)	—
Sharding	Horizontal partition across nodes	Cross-shard joins/txns, resharding, hot shards
Range partitioning	Ordered keys; range scans	Hot spots on monotonic keys

Technique	Detail	Trade-off
Hash partitioning	Uniform spread	Range scans lost
Consistent hashing	Add/remove nodes moves ~1/N keys; virtual nodes	Needs vnode tuning; hot keys remain
Hot partition	One key/shard overloaded	Key salting, split, cache, adaptive partitioning
Rebalancing	Move partitions on growth	Bandwidth, throttling, avoid stampede

4.7 Consistency models & theory

- **CAP:** during a **P**artition choose **C** (reject) or **A** (serve possibly stale). It's not "pick 2 of 3 always".
- **PACELC:** if Partition → A vs C; Else → Latency vs Consistency (Dynamo: PA/EL; Spanner: PC/EC).
- **Spectrum (strong→weak):** strict serializable → linearizable → sequential → causal → read-your-writes / monotonic reads / monotonic writes / session → eventual.
- **Consensus:** Paxos/Raft/ZAB to agree on a log → linearizable replicated state machines.
- **Distributed transactions:** 2PC (blocking on coordinator failure), 3PC (non-blocking in theory, impractical), Saga (compensations, eventual), TCC (try-confirm-cancel), Percolator/Spanner (timestamps + consensus).
- **Outbox:** write event in same local txn as state; relay publishes (solves dual-write). **Inbox:** consumer stores processed message ids (dedup). **Idempotency:** same request repeated has same effect (idempotency keys, upserts, version checks).

4.8 ★ Which database for which workload

Workload	Choice	Why	Watch out
Transactions, relations, reporting	PostgreSQL/MySQL	ACID, SQL, mature	Scale ceiling → replicas/shard
Global strongly consistent OLTP	Spanner / CockroachDB / YugabyteDB	SQL + horizontal	Latency, cost
Massive writes, known access paths	Cassandra / DynamoDB / ScyllaDB	LSM, linear scale	Model queries up front
Session/cache/leaderboards/rate limit	Redis/Memcached	µs latency, data structures	Memory cost, durability
Flexible documents, content	MongoDB/Couchbase/DocumentDB	Schema agility	Cross-doc txn, duplication
Relationship traversal	Neo4j / Neptune	Multi-hop	Sharding
Metrics/IoT	InfluxDB/TimescaleDB/Prometheus	Compression, retention	Cardinality
Full-text/log search	Elasticsearch/OpenSearch/Solr	Inverted index	Not primary store
Analytics	BigQuery/Snowflake/Redshift/ClickHouse	Columnar	Not for OLTP
Raw/ML data	S3 + Iceberg/Delta	Cheap, open	Governance
Semantic/RAG	pgvector, Pinecone, Milvus, Weaviate	ANN	Recall vs latency, re-embedding
Blobs	Object storage (S3/GCS)	Durable, cheap	High per-request latency

Part 5 – Scalability and Caching

5.1 Scalability

Always specify: **what is scaled** (requests, data, connections, fan-out), **how**, **limiting resource** (CPU, memory, disk IOPS, network, locks, single leader), **failure modes**, **cost**, **ops burden**, **max practical scale**, **assumptions**.

Concept	Definition	Limit / trade-off
Vertical scaling	Bigger machine	Simple, no code change; hardware ceiling, cost superlinear, SPOF
Horizontal scaling	More machines	Needs statelessness/partitioning; coordination, consistency
Diagonal	Scale up until cost-efficient, then out	Pragmatic mix
Stateless service	No per-user state in process (state in DB/cache/token)	Easy LB/autoscale; externalized state latency
Stateful service	Holds state (DB, sessions, WebSocket)	Needs sticky routing, replication, careful rolling deploys
Load balancing	Spread requests (see Part 2)	LB itself must be redundant
Auto-scaling / elasticity	Add/remove capacity by metrics	Lag (cold start), flapping, cost spikes, downstream not scaling
Capacity planning	Forecast + headroom	Wrong assumptions → outages or waste
Resource/connection/DB pooling	Reuse expensive resources	Pool exhaustion; size = f(DB max connections ÷ instances); PgBouncer/proxy
Replication	Copies for read scale/availability	Lag, consistency
Partitioning/sharding	Split data/load	Cross-shard ops
Caching	Part 5.2	Staleness
Batching	Group operations	Throughput↑ latency↑
Async processing	Defer work via queues	Eventual completion, ops of queues
Queue-based load leveling	Queue absorbs bursts for steady consumers	Backlog latency
Backpressure	Slow producers when consumers saturated	Needs protocol support
Rate limiting	Cap requests/client (token bucket, leaky bucket, fixed/sliding window)	Distributed counters, fairness
Throttling	Slow instead of reject	Client latency
Admission control / load shedding	Reject low-priority work to protect capacity	Needs priority classification
Read/write separation, CQRS	Different paths	Lag
Content delivery	CDN edge	Invalidation
Data locality	Compute near data / users	Placement complexity
Request coalescing (single-flight)	Merge identical in-flight requests	Prevents stampede

Concept	Definition	Limit / trade-off
Hedged requests	Send duplicate after p95 delay, take first	Extra load (~5%), idempotent only
Speculative execution	Re-run straggler tasks	Wasted compute
Work stealing	Idle workers take tasks	Used in ForkJoin pools, schedulers
Fan-out / fan-in / scatter-gather	Parallel calls then aggregate	Tail latency = max of n; partial results
MapReduce	Map shards → shuffle → reduce (Google 2004)	Disk-heavy, batch; → Spark (in-memory), Flink
Hotspot mitigation	Salting, splitting, caching, replication of hot keys	Complexity
Noisy neighbor / multi-tenancy / isolation	Shared infra among tenants	Quotas, cgroups, cell-based/bulkhead architecture, per-tenant limits; isolation vs cost

Formulas

- Little's Law: `L = λ · W` (concurrent requests = arrival rate × latency).
- Throughput per instance ≈ `cores / avg CPU time per request`.
- Amdahl: `speedup = 1 / ((1-p) + p/n)`; Universal Scalability Law adds contention and coherency costs.
- Tail at scale: if one call is slow 1% of time, a fan-out of 100 calls is slow `1-0.99^100 ≈ 63%` of the time.
- Utilization vs queueing: wait time grows ~ `ρ/(1-ρ)`; keep utilization ≲ 60–70% for latency-sensitive services.
- Pool size ≈ `QPS × avg DB time (s)` (+ headroom).

5.2 Caching

Why: locality + cost asymmetry between layers (Part 1). **Why hard:** a cache is a second copy of truth; invalidation requires knowing when truth changed, across failures and races.

Layer	What	Notes
CPU cache	L1–L3 lines	Hardware coherence
OS page/filesystem cache	Hot blocks	DBs may bypass (O_DIRECT)
DB buffer pool	Pages in RAM	Size ≈ working set
Application/local cache	In-process (Caffeine, Guava)	Fast; per-instance inconsistency
Distributed cache	Redis/Memcached cluster	Network hop; shared; cluster failure risk
CDN	Edge	Cache-Control, surrogate keys
Browser	HTTP caching, ETag/Last-Modified	Hard to invalidate (use versioned URLs)
DNS	TTL	Stale failover
Multi-level	L1 local + L2 distributed + DB	Invalidation fan-out

Write/read strategies

Strategy	Flow	Pros	Cons
Cache-aside	App checks cache, on miss reads DB & fills	Simple, resilient	Stale on races, miss penalty
Read-through	Cache loads on miss	Cleaner code	Provider coupling
Write-through	Write cache+DB synchronously	Consistent reads	Write latency
Write-back/behind	Write cache, flush later	Fast writes	Data loss on crash

Strategy	Flow	Pros	Cons
Refresh-ahead	Reload before expiry	No miss spikes	Wasted refresh
Warming	Preload	Avoid cold start	Needs knowledge

Eviction & invalidation

TTL (+ jitter), LRU, LFU (TinyLFU), FIFO, random, size/cost-aware. Invalidate by: TTL, explicit delete on write, event/CDC-driven, versioned keys, tags. Race: *read old → write DB → delete cache → stale refill*; mitigate with delayed double-delete, versioning, short TTL, leases.

Failure patterns

Problem	Cause	Fix
Stampede/thundering herd	Hot key expires, many hit DB	Single-flight/mutex, stale-while-revalidate, jittered TTL, refresh-ahead
Penetration	Requests for nonexistent keys bypass cache	Negative caching, Bloom filter, validation
Avalanche	Many keys expire together / cache dies	TTL jitter, replicas, circuit breaker, DB protection
Hot key	One key overloaded node	Local cache, key replication/splitting
Distributed cache failure	Node loss reshuffles keys	Consistent hashing, replicas, graceful DB fallback
Inconsistency	Race/invalidation lag	Accept bounded staleness, or don't cache that data

Redis-like (rich types: strings, hashes, lists, sets, sorted sets, streams, HyperLogLog, pub/sub, persistence, clustering, single-threaded commands) vs **Memcached-like** (simple strings, multi-threaded, no persistence).

Correctness effect: caching converts strong consistency into bounded staleness; don't cache money-critical read-before-write decisions without versioning.

Part 6 – Distributed Systems, Messaging and Streaming

6.1 Distributed systems

Definition: independent computers coordinating via message passing that appear as one system. **Why hard:** *partial failure*, unbounded latency, no shared clock, no global state, and you can't distinguish a slow node from a dead one.

Failure handling

Concept	Meaning	Caveat
Failure detection / heartbeats / health checks	Periodic liveness (phi-accrual detectors)	Only probabilistic
Timeouts	Bound waiting	Too short → false failure; too long → resource pile-up; propagate deadlines
Retries	Re-attempt	Need idempotency; amplify load (retry budget)
Exponential backoff + jitter	Spread retries	Avoid retry storms
Circuit breaker	Stop calling failing dependency (closed/open/half-open)	Tuning; fallbacks
Bulkhead	Isolate pools per dependency/tenant	Lower utilization
Rate limit / load shedding / graceful degradation	Protect core function	Define priorities
Leases	Time-bound ownership	Clock/pauses (GC) can expire silently
Fencing tokens	Monotonic token so stale lock holders are rejected by the resource	Required for safe distributed locks
Split-brain	Two leaders	Quorum, fencing, STONITH

Time & ordering

Concept	Detail
Physical clocks / skew	NTP error ms–100s ms; leap seconds; VM pauses
Lamport clock	Counter giving happened-before partial→total order; no concurrency detection
Vector clock	Per-node counters detect concurrent writes (Dynamo/Riak); size grows
Hybrid logical clock	Physical + logical counter (CockroachDB)
TrueTime	GPS/atomic clocks with uncertainty interval; commit-wait (Spanner)
Causality	A→B ordering preserved (causal consistency)

Agreement & coordination

Concept	Notes
Consensus	Agreement on value despite crashes; FLP: impossible in async model with 1 fault without timing assumptions

Concept	Notes
Paxos (1989/98)	Correct, hard to implement/understand; Multi-Paxos
Raft (2014)	Understandable: leader election, log replication, safety; etcd, Consul, CockroachDB
ZAB	ZooKeeper atomic broadcast
BFT (PBFT, blockchain)	Tolerates malicious nodes; 3f+1 nodes; costly
Leader election	Via consensus/lease (ZooKeeper, etcd)
Coordination services	ZooKeeper, etcd, Consul: config, locks, membership
Distributed locking	Redlock debated; prefer consensus + fencing; avoid when possible
Membership & gossip	SWIM/epidemic dissemination; eventual convergence, scalable
Quorums	Majority `N/2+1`; Dynamo `R+W>N`
Consistent hashing	Partition placement
Service discovery	Registry/DNS/mesh
Distributed scheduling	Borg/K8s scheduler, bin packing, preemption
Distributed GC	Reference counting/leases for remote objects
Distributed FS	GFS/HDFS (master + chunks, replication 3×), Ceph, Lustre
Geo-replication / multi-region	Latency vs consistency; active-active conflicts; data residency

Delivery semantics

Semantic	How	Cost
At-most-once	Send, no retry	Loss possible
At-least-once	Retry until ack	Duplicates → idempotent consumers/dedup
Exactly-once	At-least-once + dedup/transactional commit (Kafka EOS, within a system boundary)	Not truly achievable end-to-end without idempotent side effects

What not to assume

The fallacies (Part 2) + clocks are synchronized, a timeout means failure, a message is processed once, order is preserved across partitions, a lock holder is still alive, your replica is up-to-date, and one failure at a time.

6.2 Messaging & streaming

Evolution: direct request-response → message passing/IPC → message queues (IBM MQ, MSMQ, 80s–90s) → JMS/AMQP brokers (RabbitMQ, ActiveMQ) → distributed log (Kafka 2011, Pulsar, Kinesis) → stream processing (Storm → Spark Streaming → Flink, Kafka Streams) → unified batch/stream, streaming databases.

Concept	Definition	Notes
Request-response	Sync call	Temporal coupling
Message queue / work queue	Point-to-point; message removed on ack	Competing consumers
Pub/sub, topics, exchanges	Fan-out to subscribers; AMQP direct/topic/fanout/headers	Subscriber lag mgmt
Broker	Routes/stores messages	SPOF unless clustered

Concept	Definition	Notes
Log / event stream	Append-only, retained, replayable; consumers track offsets	Kafka: partitions give parallelism + per-partition order
Consumer group	Partitions divided among members; rebalancing	Max parallelism = partitions
Offset	Consumer position	Commit after processing → at-least-once
Ordering	Guaranteed within partition/key only	Choose partition key carefully
DLQ / retry queue / delayed queue / priority queue	Handle failures, backoff, scheduling, urgency	DLQ needs monitoring/redrive
Poison message	Always fails	Max-retry → DLQ
Backpressure/flow control	Prefetch limits, bounded buffers, pull model	Pull (Kafka) vs push (RabbitMQ)
Event notification	Thin event, consumer fetches data	Coupling via callback
Event-carried state transfer	Event includes data	Fewer calls; data duplication
Event sourcing	State = fold of events	See Part 3
Idempotent consumer	Dedup via id / upsert	Needed with at-least-once
Outbox/CDC pipelines	Reliable event publication	Part 4
Claim check	Store payload in blob, send reference	Large messages

Stream processing

Concept	Explanation
Event time vs processing time	Order by when it happened vs arrived
Windows	Tumbling (fixed, non-overlapping), sliding/hopping (overlap), session (gap-based)
Watermarks	"No events older than T expected" → close windows; trade completeness vs latency
Late / out-of-order events	Allowed lateness, side outputs, retractions
Stateful processing	Local state (RocksDB) + checkpoints (Flink)
Stream joins	Stream-stream (windowed), stream-table, enrichment
Exactly-once semantics	Checkpoint + transactional sinks
Micro-batching	Spark Streaming: simpler, higher latency
Lambda architecture	Batch layer + speed layer + serving; duplicate logic
Kappa architecture	Single stream path with replay; needs long retention
ETL vs ELT	Transform before load (limited warehouses) vs after (cheap scalable warehouses)
Data pipelines / orchestration	Airflow, Dagster; DAGs, retries, backfills
Durable execution	Temporal/Step Functions/Cadence: workflow state persisted, resume after crash

★ Queue vs log vs stream vs event bus

	Queue (RabbitMQ/SQS)	Log (Kafka)	Stream processor	Event bus
Retention	Until consumed	Time/size based	Derived	Varies
Replay	No	Yes	Via log	Depends
Consumption	Competing	Consumer groups, independent	Operators	Pub/sub
Ordering	Per queue (mostly)	Per partition	Per key	Weak

	Queue (RabbitMQ/SQS)	Log (Kafka)	Stream processor	Event bus
Best for	Task distribution	Event history, high throughput	Computation	Routing/integration

Event bus = logical pattern; broker = implementation (a broker can implement a bus).

★ Template deep dive: Kafka-style log

1. Definition: partitioned replicated commit log. 2. Context: LinkedIn 2011, many consumers of activity data. 3. Earlier: MQ brokers w/ delete-on-ack. 4. Problem: throughput, replay, multiple independent consumers. 5. How: producers append to partition leader; ISR replication; consumers pull by offset; sequential disk + page cache + zero-copy. 6. Components: broker, topic, partition, leader/ISR, controller (KRaft), producer, consumer group, offsets, compaction. 7. Diagram:

```
Producers -> [Topic: P0 P1 P2 (leader+replicas)] -> Consumer Group A (c1,c2,c3)
                                                 -> Consumer Group B
```

8. Use: activity tracking, CDC, event sourcing, ETL. 9. Pros: throughput, replay, decoupling. 10. Cons: ordering only per partition, rebalances, partition count hard to change, ops heavy, no per-message ack/priority. 11. Failures: consumer lag, unclean leader election data loss, rebalance storms, hot partitions. 12. Perf: MB/s–GB/s per broker; scale by partitions. 13. Consistency: acks=all + min.insync.replicas for durability. 14. Security: ACLs, TLS, SASL. 15. Ops: capacity, retention, lag monitoring. 16. Cost: disk, replication ×3, network. 17. Next: managed streaming, tiered storage, streaming DBs. 18. Still dominant. 19. Alternatives: Pulsar, Kinesis, Pub/Sub, RabbitMQ, NATS. 20. Interview: design notification pipeline, exactly-once. 21. Example: orders topic keyed by orderId. 22. Summary: high throughput + replay vs operational complexity.

Part 7 – Reliability, Availability, Security, Observability

7.1 Reliability & availability

Term	Meaning
Reliability	Correct operation over time (MTBF)
Availability	Fraction of time usable = `MTBF / (MTBF + MTTR)`
Durability	Data not lost (S3: 11 nines) — independent of availability
Resilience	Ability to recover/adapt from failure
Fault tolerance	Continue despite faults (fault ≠ failure)
Maintainability / recoverability	Ease of repair; speed of restoring
Redundancy	Extra components (N+1, N+2, 2N)
Failover	Switch to standby; detection time + data loss risk
Active-active	All serve; capacity & conflict complexity
Active-passive	Standby idle; hot (ready), warm (scaled down), cold (provision on demand)
Fault domains / AZ / region	Rack → AZ → region blast-radius boundaries
RTO / RPO	Max downtime / max data loss tolerated
SLI / SLO / SLA	Measurement / internal target / contractual with penalties
Error budget	`1 − SLO`; spend on releases; freeze when exhausted
Chaos engineering / game days / failure injection	Prove resilience (Chaos Monkey, Gremlin, FIS)
DR testing / backup validation	An untested backup is not a backup
Business continuity	Org-level plan beyond tech
Human error / operational risk	Leading cause of outages → guardrails, reviews, staged rollout, blast-radius limits
Data corruption	Checksums, scrubbing, immutable backups, versioning

Numbers

Availability	Downtime / year	/ month
99%	3.65 d	7.2 h
99.9%	8.76 h	43.8 m
99.99%	52.6 m	4.38 m
99.999%	5.26 m	26 s

- Serial dependencies multiply: `0.999 × 0.999 × 0.999 ≈ 99.7%`.
- Parallel redundancy: `1 − (1−a)^n` (two 99% replicas → 99.99%, if independent and failover perfect).
- Correlated failures (same AZ, same bug, same config push) break independence assumptions.
- DR tiers: backup/restore (RTO hours, RPO hours) → pilot light → warm standby → multi-site active-active (RTO≈0).

7.2 Security

Concept	Risk mitigated	Trade-off
Threat modeling (STRIDE), attack surface	Unknown risks	Time
CIA triad	Frames goals	—
AuthN / AuthZ / Accounting	Identity / permissions / audit	Friction
RBAC / ABAC / ReBAC (Zanzibar)	Access policy scales	Role explosion / policy complexity
Least privilege, defense in depth, zero trust	Breach blast radius	Operational overhead
Secrets/key management (Vault, KMS, HSM), rotation	Leaked credentials	Availability dependency on KMS
Encryption at rest/in transit/in use (confidential computing, homomorphic)	Disk theft, sniffing, memory inspection	CPU, key mgmt
Symmetric/asymmetric, hashing (bcrypt/argon2 for passwords, salted), signatures, certificates, PKI, TLS	Confidentiality/integrity/identity	Expiry/rotation outages
OAuth 2.0 (delegated authorization), OIDC (authentication layer), JWT (stateless signed token)	Password sharing, SSO	JWT revocation difficulty, token theft; use short TTL + refresh
Sessions/cookies (HttpOnly, Secure, SameSite)	State over stateless HTTP	Session store scaling
CSRF / CORS / XSS	Browser attacks	CSP, tokens, output encoding
SQL injection	Query tampering	Parameterized queries
SSRF / RCE	Internal access / code exec	Egress filtering, input validation, sandboxing, metadata-service hardening
DDoS	Availability attack	CDN/anycast/scrubbing, rate limits; cost
WAF, API security (authN, schema validation, quotas, BOLA checks)	App-layer attacks	False positives
Segmentation, mTLS, service identity (SPIFFE)	Lateral movement	Cert lifecycle
Supply chain (SBOM, signing, pinned deps, SLSA), dependency & container security (minimal images, scanning, non-root), runtime security (seccomp, Falco)	Compromised components	Build friction
Audit logging, compliance (SOC2, ISO27001, PCI, HIPAA, GDPR)	Accountability	Cost, retention vs privacy tension
Privacy: PII, masking, tokenization, residency, retention, secure deletion (crypto-shredding)	Exposure, legal	Complexity in backups/event logs

7.3 Observability & operations

Monitoring = known questions answered with predefined metrics/alerts ("is CPU > 80%?"). **Observability** = ability to ask new questions about unknown failure from telemetry (logs + metrics + traces + profiles with high-cardinality context). **Logging** = event detail; monitoring = watch known signals.

Concept	Notes
Logging	Structured (JSON), levels, centralized (ELK/Loki), no PII/secrets
Metrics	Counter, gauge, histogram, summary; percentiles (p50/p95/p99) — never average latency; RED (rate/errors/duration) & USE (utilization/saturation/errors), Four Golden Signals
Tracing	Spans, trace/correlation IDs, context propagation (W3C traceparent), OpenTelemetry standard; sampling (head/tail)

Concept	Notes
Profiling	CPU/heap/continuous profiling (pprof, async-profiler, JFR)
Alerting	Alert on symptoms/SLO burn rate, not causes; reduce noise
Dashboards	Per-service + business KPIs
Health checks	Liveness (restart me), readiness (send traffic), startup
Synthetic / RUM	Scripted probes vs real-user measurements
High-cardinality / cost	Label explosion kills TSDBs; sample, aggregate, retention tiers
Telemetry privacy	Scrub PII
Incident response	Detect → triage → mitigate → communicate → resolve → postmortem
On-call, runbooks, blameless postmortems, SRE	Sustainable ops, learning culture
Deployment tracking / auditability	Correlate changes with incidents (most outages follow change)
Debugging distributed systems	Trace → logs by ID → metrics → hypothesis; reproduce with replay

Part 8 – Deployment, Cloud, Containers, Serverless, Edge

8.1 Infrastructure evolution

```
Physical servers/bare metal -> Virtual machines (hypervisors) -> IaaS cloud (EC2, 2006)
 -> PaaS (Heroku, App Engine) -> Containers (Docker 2013) -> Orchestration (Kubernetes 2014)
 -> Service mesh -> Serverless/FaaS & managed services -> Edge / multi-region -> Platform engineering
```

Stage	Pressure	Solves	New problem
Physical/bare metal	—	Max performance, control	Weeks to provision, low utilization, snowflake servers
VMs/hypervisor	Cost/utilization	Consolidation, isolation	Overhead, big images, slow boot
IaaS	Capex & lead time	Elastic, pay-as-you-go	Still manage OS/patching
PaaS	Ops burden	Push code	Constraints, lock-in
Containers (namespaces+cgroups, images, registries)	Env drift, density	Portable, fast, immutable packaging	Shared kernel (weaker isolation), orchestration needed; use microVMs (Firecracker/gVisor) when strong isolation required
Kubernetes	Managing many containers	Scheduling, self-healing, rolling updates, service discovery, ingress, autoscaling	Steep complexity, cost, YAML sprawl
Sidecar/service mesh (Istio, Linkerd; control plane + data plane)	Cross-cutting network concerns (mTLS, retries, telemetry)	Uniform policy w/o app code	Latency, resource overhead, debugging → ambient/eBPF meshes
Serverless	Idle cost, scaling ops	Scale to zero	Cold starts, limits, lock-in, hard local testing
Managed services	Undifferentiated heavy lifting	Less ops	Cost at scale, lock-in, opaque limits

Containers vs VMs: share host kernel (fast, dense, weaker isolation) vs own kernel (strong isolation, heavier).
Kubernetes vs serverless: control + portability + steady load vs zero-ops + spiky/low load.

8.2 Delivery & operations

Concept	Notes
IaC (Terraform, CloudFormation, Pulumi)	Reproducible infra; state drift, secrets in state
Configuration management (Ansible, Chef, Puppet)	Mutable convergence → replaced by immutable infrastructure (replace, don't patch)
CI / continuous delivery / continuous deployment	Delivery = deployable always; deployment = auto to prod
Blue-green	Two environments, instant switch/rollback; double cost, DB compat
Rolling	Replace gradually; mixed versions co-exist
Canary	Small % first, auto-analysis; needs good metrics

Concept	Notes
A/B	Experimentation (business), not safety
Feature flags	Decouple deploy from release; flag debt
Shadow traffic	Mirror prod traffic to new version; side effects must be disabled
Rollbacks	Easy for stateless, hard with schema/data changes → forward-fix
DB deployment	Expand-and-contract: add (compatible) → dual write/backfill → switch reads → remove old
GitOps (Argo CD/Flux)	Git as source of truth; reconciliation
Platform engineering / IDP	Paved roads, self-service, reduce cognitive load; product mindset
IaaS/PaaS/SaaS/FaaS/CaaS	Responsibility ladder
Regions/AZs	Isolation units; cross-AZ traffic costs
Multi-cloud / hybrid	Resilience/regulation/negotiation vs lowest-common-denominator and complexity
Vendor lock-in	Weigh switching cost vs speed gained
FinOps / cost optimization	Tagging, rightsizing, reserved/spot, autoscaling, storage tiering, egress minimization

8.3 Edge & global systems

- **CDN → edge compute** (Cloudflare Workers, Lambda@Edge): logic at POPs; limited runtime/state → edge KV, Durable Objects.
- **Global load balancing:** GeoDNS, anycast, latency routing.
- **Multi-region patterns:** active-passive (simple, RPO>0), active-active (needs conflict handling or region-pinned data, CRDTs, global DB), cell-based architecture (blast-radius isolation), data residency partitioning.
- **IoT/edge:** intermittent connectivity → local processing, MQTT, store-and-forward.

Part 9 – Design Patterns (Problem · Solution · Consequences · Alternatives)

9.1 GoF object-oriented patterns

Pattern	Problem	Solution	Consequences	Alternative
Singleton	Need exactly one instance	Private ctor + global access (enum/holder in Java)	Hidden global state, hard tests, per-JVM only	DI-managed single scope
Factory method	Creation varies by subclass	Subclass decides product	More classes	Static factory, DI
Abstract factory	Families of related objects	Factory of factories	Hard to add product kinds	DI container
Builder	Many optional ctor args	Step-wise construction	Verbosity	Named params, records
Prototype	Costly creation	Clone existing	Deep-copy pitfalls (see `cloning/`)	Copy constructors
Adapter	Incompatible interface	Wrap & translate	Extra layer	Anti-corruption layer
Bridge	Abstraction × implementation explosion	Separate hierarchies	Upfront design	Strategy
Composite	Tree of part/whole	Uniform component interface	Overly general	—
Decorator	Add behavior dynamically	Wrap same interface	Many small wrappers (Java I/O streams)	Inheritance, AOP
Facade	Complex subsystem	Simple entry API	Can hide needed features	API gateway (distributed analog)
Flyweight	Many similar objects	Share intrinsic state	Complexity	Interning, caching
Proxy	Control access (lazy, remote, security)	Surrogate object	Indirection	Sidecar (distributed analog)
Chain of responsibility	Multiple handlers	Pass along chain	Unhandled requests	Middleware pipelines/filters
Command	Encapsulate request	Object with execute/undo	Class proliferation	Lambdas; queues of commands
Interpreter	Grammar evaluation	AST evaluation	Slow, complex grammars	Parser generators
Iterator	Traverse w/o exposing structure	Iterator object	—	Streams
Mediator	N-to-N coupling	Central coordinator	God object	Event bus
Memento	Snapshot/undo	Opaque state capture	Memory	Event sourcing
Observer	One-to-many notification	Subscribe/notify	Leaks, ordering	Pub/sub, reactive streams
State	Behavior per state	State objects	More classes	State machines
Strategy	Swap algorithms	Interchangeable objects	Client chooses	Lambdas

Pattern	Problem	Solution	Consequences	Alternative
Template method	Fixed skeleton, varying steps	Base class steps	Inheritance coupling	Strategy/composition
Visitor	New operations on stable hierarchy	Double dispatch	Hard to add types	Pattern matching/sealed types

9.2 Enterprise / application patterns

Pattern	Problem	Solution	Consequences / alternative
Repository	Domain coupled to persistence	Collection-like abstraction	Leaky for complex queries; Spring Data
Unit of work	Track changes & commit atomically	Collect changes, flush in txn	Session lifetime bugs
Data mapper vs Active record	Where mapping lives	Separate mapper (Hibernate) vs record knows DB (Rails)	Mapper: decoupled, heavier; AR: simple, coupled
Dependency injection	Hard-wired deps	Container supplies deps	Magic/config; constructor injection
Service layer / Domain model / Transaction script	Where logic lives	Orchestration layer; rich objects; procedural script	Script for simple CRUD, model for complex domains
Specification	Reusable business rules	Composable predicates	Verbosity
Front controller / MVC	Central request handling	Single entry + dispatch	—
API gateway / BFF	Many clients/services	Edge façade	SPOF, logic creep
Strangler fig	Migrate legacy incrementally	Route gradually to new system	Dual-run cost; vs big-bang rewrite
Anti-corruption layer	Legacy model leaks into new	Translation layer	Maintenance
Circuit breaker	Cascading failure	Trip after failures	Tuning; with fallback
Bulkhead	One failure exhausts all resources	Isolated pools	Underuse
Retry / Timeout / Retry with backoff	Transient faults	Re-attempt bounded with jitter	Storms; needs idempotency
Rate limiter / Throttling	Overload/abuse	Token bucket etc.	Distributed counting
Cache-aside	Read-heavy	Lazy load cache	Staleness
Sidecar / Ambassador / Adapter service	Cross-cutting/ outbound proxying / protocol translation	Co-located helper process	Resource overhead
Saga	Cross-service business txn	Local txns + compensations (orchestration vs choreography)	Eventual consistency, compensation design; alt: 2PC
TCC	Reservation-style distributed txn	Try/Confirm/Cancel	Business-level undo
Two-phase commit	Atomic multi-resource commit	Prepare then commit via coordinator	Blocking, latency; alt: saga
Outbox / Inbox / Transactional messaging	Dual-write problem / duplicates	Event in same txn; dedup table	Relay latency
Idempotency key	Retried writes	Client key stored w/ result	Storage/TTL

Pattern	Problem	Solution	Consequences / alternative
Competing consumers	Scale processing	Workers share queue	Ordering lost
Claim check	Large payloads	Store blob, pass ref	Extra store
Dead-letter queue / Priority queue	Poison/urgent messages	Side queue / ranked	Starvation of low priority
Fan-out / Fan-in / Scatter-gather	Parallelism	Broadcast then aggregate	Tail latency
Leader election	Single coordinator	Lease/consensus	Split-brain risk
Sharding / Consistent hashing	Data/load distribution	Partition keys	Resharding
Materialized view	Expensive reads	Precomputed view	Freshness
CQRS / Event sourcing / Event notification / Event-carried state transfer	See Parts 3 & 6		
Hedged requests	Tail latency	Duplicate after delay	Extra load

Part 10 – System Designs Mapped to Concepts

Each entry: **Core requirements → Key design → Concepts used → Main risks.** Estimates are illustrative.

#	System	Key design	Concepts	Risks
1	URL shortener	Base62 of unique id (counter/Snowflake/KGS), KV store, 301/302 redirect, cache hot links; read:write ≈100:1	KV DB, sharding by key, cache-aside, CDN, rate limit	Key collisions, hot links, abuse
2	Pastebin	Blob in object store, metadata in DB, TTL expiry	Object storage, lifecycle policies, CDN	Abuse/malware, size limits
3	File storage (Dropbox)	Chunking (4 MB), dedup by hash, metadata DB, sync via change log, resumable upload	Content-addressed storage, CDC, delta sync, conflict resolution	Conflicts, consistency of metadata vs blobs
4	Object storage (S3)	Metadata/index layer + data nodes, erasure coding (e.g., 10+4), 11-nines durability	Replication vs EC, consistent hashing, scrubbing	Repair traffic, hot buckets
5	Distributed FS (GFS/HDFS)	Master (namespace) + chunkservers, 3× replication, large sequential I/O	WAL, leases, rack-aware placement	Master SPOF → HA/federation, small files
6	Key-value store (Dynamo)	Consistent hashing w/ vnodes, N/R/W quorums, vector clocks/LWW, hinted handoff, gossip, Merkle trees, LSM	Leaderless replication, anti-entropy	Conflicts, tuning
7	Distributed cache	Sharded Redis/Memcached, consistent hashing, replication, eviction	Cache patterns, hot keys	Stampede, failover
8	Search engine	Crawl → index (inverted, sharded) → rank (TF-IDF/BM25/PageRank/ML) → serve (scatter-gather)	Lucene segments, tiered index, caching	Freshness vs cost, tail latency
9	Web crawler	URL frontier (priority + politeness queues per host), dedup (Bloom/simhash), robots.txt, DNS cache	Queues, partitioning by host, rate limit	Traps, duplicates
10	Notification system	Event → preference/dedupe → per-channel queues (push/SMS/email) → provider → status	Pub/sub, DLQ, idempotency, rate limits, priority queues	Provider outages, duplicates, spam
11	Email system	SMTP ingest, queue, spam filter, mailbox store (per-user), IMAP/index	Queues, search index, reputation/SPF/DKIM/DMARC	Deliverability
12	Chat	WebSocket gateways, session registry (user→gateway), message log per conversation, offline store+push, ordering via per-chat sequence	Stateful connections, partition by conversation, fan-out, presence TTL	Reconnect storms, ordering, group fan-out
13/14	Social/news feed	Fan-out-on-write (precompute timelines) vs on-read; hybrid for celebrities; ranking	Caches, queues, sharding, hot keys	Celebrity problem, ranking freshness
15	Video streaming	Upload→transcode DAG (multiple bitrates/codecs)→segments (HLS/DASH)→CDN, adaptive bitrate	Object storage, queues, CDN/edge, DRM	Egress cost, popular-content spikes
16	Music streaming	Similar + licensing, offline sync, recommendations	CDN, caches, recommender	Rights/regions

#	System	Key design	Concepts	Risks
17	Ride-sharing	Geo-index (geohash/S2/H3), location stream (ingest every ~4 s), matching, ETA, surge pricing, trip state machine	Streams, in-memory geo store, sagas	Hot cities, GPS noise
18	Food delivery	Orders state machine, restaurant/courier matching, ETA, payments	Sagas, event-driven, geo	Multi-party consistency
19	Payment system	Idempotency keys, double-entry ledger, state machine, reconciliation, PSP integration, async webhooks	ACID DB, outbox, audit, PCI tokenization	Double charge, unknown outcomes (timeouts)
20	Banking	Ledger with immutable entries, strong consistency, batch settlement, regulatory audit	Serializable txns, event sourcing, DR with RPO≈0	Compliance
21	E-commerce	Catalog (search+cache), cart (KV), inventory (reservation), orders (saga), payments	Microservices, CQRS, search index	Overselling, flash sales
22/23	Ticket/hotel booking	Seat/room hold with TTL, optimistic locking/ `SELECT FOR UPDATE`, waiting room under spikes	Concurrency control, queues, admission control	Double booking
24	Calendar	Events with recurrence rules (RRULE), time zones, free/busy, invites, sync (CalDAV)	Interval indexing, notifications	Time-zone/DST bugs
25	Online judge	Submission queue → sandboxed workers (containers/seccomp/cgroups) → verdict	Queues, isolation, resource limits	Malicious code
26	Metrics platform	Agents → ingest (Kafka) → TSDB (compression, downsampling) → query/alert	Streams, high cardinality, retention tiers	Cardinality explosion
27	Logging platform	Agents → buffer → index/object store → query; sampling, retention	Backpressure, hot/warm/cold tiers	Cost, PII
28	Analytics platform	Events → stream/batch → lakehouse → OLAP/BI	ETL/ELT, columnar, partitioning	Data quality
29	Recommendation	Candidate generation (ANN/CF) → ranking (ML) → re-ranking; feature store; offline+online	Vector search, caches, A/B tests	Feedback loops, cold start
30	Ad serving	Auction (<100 ms), targeting, budget pacing, click logging, attribution	Low latency, in-memory, streaming aggregation	Fraud, budget over-delivery
31	Fraud detection	Streaming features + rules + ML scoring within ms, case management	Stream processing, feature store, human review loop	False positives
32	Real-time bidding	~100 ms end-to-end, massive QPS, fan-out to bidders, timeouts	Hedging, load shedding, in-memory	Tail latency
33	IoT platform	MQTT broker, device registry/shadow, OTA, time-series, rules engine	Edge, store-and-forward, cert-based auth	Device fleet security
34	Collaboration (Docs)	OT or CRDT, WebSocket, presence, version history	Causal ordering, snapshots+ops	Merge semantics
35	Workflow engine	Durable state machine, timers, retries, history	Durable execution, event sourcing	Determinism, versioning
36	Job scheduler	Cron/delay queue, leader election, leasing, exactly-once-ish	Leases, fencing, DB-based queues	Duplicate runs, clock skew

#	System	Key design	Concepts	Risks
37	Distributed task queue	Brokers, workers, visibility timeout, retries, DLQ	Competing consumers, idempotency	Poison tasks
38	API platform	Gateway, authN/Z, quotas, billing, developer portal, versioning, SDKs	Rate limiting, observability	Breaking changes
39	Multi-tenant SaaS	Pool vs silo vs bridge; tenant id everywhere; per-tenant quotas	Isolation, noisy neighbor, cell architecture	Data leaks across tenants
40	CMS	Content store, versioning, workflow, CDN, search, templating	Caching/invalidation, RBAC	Cache purge
41	Learning platform	Content delivery, progress tracking, assessments, video	CDN, event tracking	Exam-time spikes
42	AI inference platform	Request router, batching (continuous batching), GPU pools/autoscale, model registry, KV cache, quantization, canary models, token streaming (SSE)	Queues, admission control, cost control	GPU scarcity, cold starts, tail latency
43	ML feature store	Offline (lake) + online (KV) stores, point-in-time correct joins, consistent definitions	Streaming + batch, TTL	Training/serving skew
44	Vector search	Embeddings → ANN index (HNSW/IVF-PQ) sharded, hybrid with keyword (BM25), metadata filters	Sharding, memory planning, re-indexing	Recall vs latency, embedding model changes
45	RAG system	Ingest → chunk → embed → index; query → rewrite → retrieve (hybrid) → rerank → prompt → LLM → cite/guardrails; eval loop	Vector DB, caches, ACLs per document, observability, cost	Hallucination, stale docs, prompt injection, data leakage

★ Full example: URL shortener

```
Client -> CDN/LB -> API (create/redirect) -> Cache (Redis) -> KV/SQL (sharded by hash(code))
                        \-> ID service (range-allocated counters) -> Base62
                        \-> Analytics events -> Kafka -> OLAP
```

- 100 M new URLs/month ≈ 40 writes/s; 100:1 reads ≈ 4,000 reads/s avg, ×5 peak = 20 K/s.
- 7-char base62 = 62^7 ≈ 3.5 T codes. 500 B/record × 100 M × 12 × 5 yrs ≈ 3 TB.
- Cache top 20% → ~80% hit rate; redirects 302 if analytics needed, 301 if cacheable forever.

Part 11 – Trade-offs, Confused Concepts, Numerical Reasoning

11.1 Trade-off dimensions

Latency · Throughput · Scalability · Availability · Consistency · Durability · Fault tolerance · Cost · Complexity · Security · Operability · Maintainability · Developer productivity · Time to market · Vendor dependence · Data correctness · RTO · RPO · Geographic scope.

Decision	Latency	Throughput	Availability	Consistency	Cost	Complexity
Add cache	↓↓	↑	↑ (shields DB)	↓ (stale)	+	+
Async queue	↑ end-to-end	↑	↑	eventual	+	++
Sync replication	↑ writes	~	↑ durability	strong	+	+
Async replication	~	~	RPO>0	eventual	+	+
Sharding	~	↑↑	partial-failure	cross-shard weak	++	+++
Microservices	↑ (hops)	↑ per service	isolation ↑	eventual	++	+++
Multi-region active-active	↓ local	↑	↑↑	conflicts/↑ latency for strong	+++	++++
Serverless	cold starts	burst ↑	managed	n/a	pay-per-use	lock-in

11.2 Commonly confused concepts

Pair	Distinction
Process vs thread	Isolated address space vs shared memory within process
Concurrency vs parallelism	Dealing with many things vs doing many at once
Sync vs async	Caller waits for completion vs continues/gets notified
Blocking vs non-blocking	Thread parks on call vs call returns immediately (can be sync+non-blocking polling)
Stateful vs stateless	Server remembers client context vs each request self-contained
Vertical vs horizontal	Bigger node vs more nodes
Replication vs partitioning	Copies of same data vs different subsets (usually combined)
Sharding vs partitioning	Sharding = partitioning across nodes; partitioning may be within one node
Queue vs stream	Consume-and-delete work distribution vs retained ordered replayable log
Event bus vs message broker	Pattern/channel concept vs server software implementing it
Cache vs database	Disposable derived copy for speed vs source of truth with durability
OLTP vs OLAP	Many small transactional ops vs few large analytic scans
Strong vs eventual consistency	All see latest write vs converge later
Availability vs durability	System answers vs data persists (can be down yet durable)
Reliability vs resilience	Consistently correct vs recovers from adversity

Pair	Distinction
Latency vs throughput	Time per request vs requests per time (batching trades one for other)
REST vs RPC	Resources + uniform verbs vs procedure calls
REST vs GraphQL	Server-defined shapes vs client-defined queries
TCP vs UDP	Reliable ordered stream vs best-effort datagrams
HTTP/1.1 vs 2 vs 3	Text/serial per connection → multiplexed over TCP → multiplexed over QUIC
Proxy vs reverse proxy	Acts for clients vs acts for servers
LB vs API gateway	Distributes traffic vs adds API management (auth, quota, transformation); often combined
Service mesh vs API gateway	East-west service-to-service vs north-south edge
Monolith vs modular monolith	Unstructured vs enforced module boundaries (single deploy)
Monolith vs microservices	Single deployable vs many; ACID vs eventual
SOA vs microservices	Enterprise-wide, ESB, coarse vs decentralized, fine-grained, smart endpoints/dumb pipes
Containers vs VMs	Process isolation sharing kernel vs full virtual hardware
Kubernetes vs serverless	You operate cluster vs provider abstracts it
Batch vs stream	Bounded vs unbounded data
ETL vs ELT	Transform before vs after load
Warehouse vs lake vs lakehouse	Structured/curated vs raw/any vs lake with ACID table format
AuthN vs AuthZ	Who are you vs what may you do
Encryption vs hashing	Reversible with key vs one-way
Logging vs monitoring vs observability	Records vs known-signal watching vs ability to explore unknowns
DR vs HA	Recover from site loss vs survive component failures without downtime
Backup vs replication	Point-in-time protection from corruption/deletion vs live copy that replicates mistakes
Exactly-once vs idempotent	Delivery guarantee (hard) vs effect-level safety under duplicates (practical)
2PC vs Saga	Atomic, blocking, locks vs eventually consistent with compensations
CQRS vs event sourcing	Split read/write models vs store events as truth; independent, often combined
Event sourcing vs EDA	Persistence strategy vs integration/communication style

11.3 Numerical reasoning cheat sheet

Topic	Formula / example
QPS	`DAU × actions/day / 86,400`; 10 M DAU × 10 req = 1,157 QPS avg; peak ≈ 2–10×
Bandwidth	`QPS × payload`; 5,000 QPS × 200 KB = 1 GB/s (8 Gbps)
Storage	`records × size × retention × replication`; 1 B × 1 KB × 3 = 3 TB
Replication overhead	N× storage; erasure coding (k+m)/k (10+4 = 1.4×)
Cache hit ratio	`avg latency = h·Tc + (1−h)·Td`; h=0.95, Tc=1 ms, Td=20 ms → 1.95 ms
DB capacity	Single Postgres node ~ thousands–tens of thousands simple TPS; shard when write-bound/size-bound
Queue/lag	Backlog growth = `(in − out) × t`; drain time = backlog / (out − in); lag = latest offset − committed offset

Topic	Formula / example
Latency budget	Sum of hops; SLO 200 ms across 5 serial calls → ~40 ms each
Availability	See Part 7
Probability of failure	`1 - (1-p)^n`; 1,000 disks at 2% AFR → ~20 failures/year
Recovery	RTO = detect + decide + failover + warm-up; RPO = replication lag / backup interval
Headroom	Provision for peak ×1.3–2 and N+1 (survive AZ loss: with 3 AZs run at ≤ 66%)
Traffic growth	`V·(1+g)^t`; 10%/month → ×3.1/year
Cost	`instances × $/h × 730`, plus egress ($/GB), storage tiers, requests, people

Part 12 – Decision Frameworks, Failed/Oversold Ideas, Failure Cases

12.1 Decision trees (condensed)

```
Monolith or microservices?
 team < ~20 and domain unclear?            -> Modular monolith
 independent scaling/deploy/team autonomy
   needed AND platform/observability ready? -> Microservices (start from bounded contexts)
 else                                       -> Modular monolith, extract hot spots later

SQL or NoSQL?
 relations, ad-hoc queries, transactions?   -> SQL
 extreme write scale, known key access?     -> Wide-column/KV
 flexible nested docs?                      -> Document (or SQL JSONB)
 traversals?                                -> Graph ; text? -> Search ; embeddings? -> Vector
 default when unsure                        -> PostgreSQL

Strong vs eventual consistency?
 money/inventory/uniqueness invariants?     -> Strong (single-leader/consensus)
 stale reads tolerable, high availability?  -> Eventual (+ read-your-writes where needed)

Queue or stream?
 distribute tasks, delete after done        -> Queue
 many consumers, replay, ordering by key    -> Log/stream

Sync or async?
 caller needs result now & dependency fast  -> Sync (with timeout/circuit breaker)
 long work / spiky / fan-out                -> Async

Cache or database?  Derived, recomputable, read-heavy? -> Cache; source of truth? -> DB
Replication or sharding? Read-bound -> replicas; write/data-size-bound -> shard
REST/GraphQL/gRPC? public/simple -> REST; many varied clients -> GraphQL; internal/perf -> gRPC
Containers or VMs? density/portability -> containers; hard isolation/legacy OS -> VMs (or microVMs)
Kubernetes or serverless? steady/complex/portable -> K8s; spiky/small team/event -> serverless
Single or multi-region? global users or RTO/RPO beyond single-region -> multi (start active-passive)
Active-active or active-passive? need ~0 RTO and can resolve conflicts -> active-active
Batch or real-time? latency requirement in seconds & value decays fast -> streaming
Build or buy? differentiating core -> build; commodity -> buy
Managed or self-hosted? small team -> managed; scale/cost/compliance justify ops -> self-host
Transaction or saga? same DB -> transaction; across services -> saga/outbox
Centralized or distributed? Distribute only when scale/availability/org/latency forces it.
```

12.2 Failed, oversold, or niche ideas

Idea	Attempted to solve	Why attractive	Why faded	Replaced by	Still valid?
CORBA/DCOM	Language-neutral remote objects	Location transparency	Complexity, interop, firewalls	REST/SOAP/gRPC	Rare
EJB 2 / heavy J2EE	Enterprise components	Standardization	Boilerplate	Spring, POJOs	Legacy
SOAP/WS-* + ESB	Enterprise integration	Standards, governance	Verbosity, central bottleneck	REST, brokers, microservices	B2B legacy
Object-oriented DBs	Impedance mismatch	Natural model	No ecosystem	ORM, document DBs	Niche

Idea	Attempted to solve	Why attractive	Why faded	Replaced by	Still valid?
Early "NoSQL replaces SQL"	Scale	Horizontal scale	Lost joins/transactions	SQL returned (NewSQL, JSON in SQL)	Both coexist
MapReduce (Hadoop)	Batch at scale	Simple model	Disk-heavy, slow iteration	Spark/Flink	Legacy
3PC	Non-blocking commit	Avoid 2PC block	Assumes synchronous network	Consensus-based commit	Academic
Distributed objects/transparency	Hide the network	Developer ease	Violates fallacies	Explicit async APIs	No
Blockchain-for-everything	Trustless data	Hype	Throughput, cost; DB suffices without adversaries	Databases	Niche (multi-party trust)
Microservices-first for small teams	Scale/agility	Fashion	Ops overhead	Modular monolith	Misapplied
Nano-services	Max granularity	Reuse	Latency, cost	Merge services	No
Shared database between services	Easy integration	Simplicity	Coupling	DB per service/APIs	Short-term
Distributed transactions everywhere (XA)	Atomicity	Familiar	Blocking, throughput	Sagas/outbox	Limited
Server push in HTTP/2	Faster loads	Fewer RTTs	Cache complexity	103 Early Hints/preload	Removed from Chrome
Pure lambda architecture	Batch+real-time	Accuracy	Duplicate code	Kappa/unified engines	Some cases
Unmanaged self-built service meshes	Uniform networking	Features	Resource/ops cost	Ambient/eBPF, library approaches	Selective
Popularity ≠ superiority: judge by constraints.					

12.3 Failure scenarios: detect · mitigate · recover · prevent

Scenario	Detect	Mitigate	Recover	Prevent
Database overload	Latency/pool saturation, slow-query log	Shed load, kill queries, read replica, cache	Scale, index, restore capacity	Query review, pools, rate limits, load tests
Cache stampede	DB QPS spike after expiry	Single-flight, serve stale	Warm cache	TTL jitter, refresh-ahead
Hot partition	Skewed per-shard metrics	Split/salt, cache, throttle	Rebalance	Key design, adaptive sharding
Message duplication	Dup ids/metrics	Idempotent consumers	Dedup/repair	Idempotency keys, EOS
Message loss	Gaps in sequence, audits	Replay from log	Reprocess	acks=all, durable queues, outbox
Consumer lag	Lag metric/alert	Scale consumers, skip poison	Drain backlog	Capacity plan, partitions, DLQ
Network partition	Timeouts, quorum loss	Minority stops writes	Heal + reconcile	Quorums, tested CAP choice
Split-brain	Two leaders	Fencing, STONITH	Reconcile writes	Consensus-based election

Scenario	Detect	Mitigate	Recover	Prevent
Region outage	Synthetic probes, provider status	Fail over traffic	Restore/failback	Multi-region, DR drills
Clock skew	NTP offset alerts	Use monotonic/logical clocks	Fix time sync	Avoid wall-clock ordering, fencing tokens
Cascading failure	Error spread across tiers	Circuit breakers, shed load	Staged restore	Bulkheads, timeouts, capacity
Retry storm	Request multiplication	Disable retries, rate limit	Ramp traffic	Backoff+jitter, retry budgets
Thundering herd	Synchronous reconnects/expiry	Staggering	Gradual ramp	Jitter, queueing
Data corruption	Checksums, audits	Stop writes, isolate	PITR/backups	Checksums, immutable backups, validation
Schema incompatibility	Deploy errors	Rollback/feature flag	Forward-fix	Expand-contract, contract tests
Failed deployment	Canary metrics	Auto-rollback	Redeploy	Canary, flags, staging parity
Certificate expiration	Expiry monitors	Replace cert	Rotate	Auto-renewal (ACME), alerts
Credential leakage	Secret scanning, anomaly	Revoke/rotate	Forensics	Vault, short-lived creds, scanning
DDoS	Traffic anomaly	Scrubbing/CDN, rate limit	Block sources	Anycast, WAF, autoscale caps
Storage exhaustion	Disk alerts	Free/expand, retention	Restore service	Quotas, lifecycle
Memory leak	RSS growth, GC time	Restart/rolling	Fix leak	Profiling, limits
Deadlock	Thread dumps, hung txns	Kill victim	Restart	Lock ordering, timeouts
Queue backlog	Depth/age	Add workers, shed	Drain	Autoscale on lag
Incorrect failover	Data divergence	Freeze writes	Reconcile	Fencing, drills, automation limits
Partial rollback	Mixed versions	Complete forward/back	Reconcile data	Backward-compatible changes
Inconsistent replicas	Merkle/anti-entropy diff	Read repair	Resync	Quorums, checksums
Vendor outage	Dependency health	Fallback/degrade	Wait/failover	Multi-vendor critical paths, graceful degradation

Part 13 – Data/AI Systems, Emerging Approaches, Final Evolution Map, Audit

13.1 Data-intensive & AI-enabled systems (P)

Concept	Pressure → response	Trade-offs
Big data batch (Hadoop/MapReduce → Spark)	Data beyond one machine → move compute to data	Latency, ops
Stream processing (Flink/Kafka Streams)	Need seconds-level insight	State mgmt, late data
Lakehouse (Iceberg/Delta/Hudi)	Warehouse + lake silos	Maintenance (compaction)
Data mesh / data products	Central data team bottleneck	Governance federation
Feature store	Training/serving skew	Dual stores
ML pipeline (train → validate → registry → deploy → monitor), MLOps	Reproducibility	Drift monitoring, retraining cost
Model serving	Online (low-latency, autoscale GPU), batch, streaming; dynamic/continuous batching; caching; quantization; model sharding (tensor/pipeline parallel)	GPU cost/latency
Vector DB / embeddings	Semantic similarity	Approximate recall
RAG	LLM knowledge cutoff/hallucination → retrieval grounding	Retrieval quality, ACL leakage, prompt injection
LLM gateway/agents	Provider routing, caching, guardrails, tool use, evals, token budgets	Non-determinism, cost, observability of prompts
Training infra	Distributed data/model parallel, checkpointing, fault tolerance, high-bandwidth interconnects	Cost, failures at scale
Data governance for AI	Lineage, consent, PII filtering	Compliance overhead

13.2 Emerging / current approaches (Q)

Platform engineering & IDPs · cell-based architecture · WebAssembly at edge/server · eBPF observability/networking · ambient service mesh · durable execution · local-first/CRDT apps · streaming databases (Materialize, RisingWave) · tiered/disaggregated storage (compute-storage separation) · confidential computing · serverless containers · multi-agent LLM systems · policy-as-code (OPA) · FinOps/GreenOps · supply-chain security (SLSA, sigstore) · zero-trust by default.

13.3 A. Timeline

Era	Milestones
1950s–60s	Batch, mainframes, time-sharing, IMS, packet switching idea
1970s	Unix, Relational model (1970), TCP/IP, Ethernet, SQL, pipes & filters

Era	Milestones
1980s	PCs, client-server, RPC, Erlang, NFS, DNS (1983)
1990s	WWW, HTTP/1.0–1.1, 3-tier, CORBA, Java, SSL, GoF patterns (1994), CDNs, MQs
2000s	REST, SOA/SOAP, Google GFS/MapReduce/Bigtable, Dynamo, AWS (2006), Hadoop, virtualization, CAP
2010s	NoSQL wave, Kafka, Docker, Kubernetes, microservices, Spanner, Raft, serverless, GraphQL, gRPC, SRE, DevOps, HTTP/2
2020s	HTTP/3/QUIC, service mesh maturity, lakehouse, platform engineering, edge compute, vector DBs, LLM/RAG infrastructure, eBPF, WASM

13.4 B. Dependency graph (condensed)

```
Hardware -> OS -> Processes/Threads -> Concurrency control
Hardware(disk/RAM) -> Storage engines -> DBs -> Transactions -> Replication -> Consensus
Networking -> RPC/HTTP -> Client-server -> Web -> Load balancing -> Caching/CDN
Replication + Partitioning + Network unreliability -> Distributed systems -> CAP/consistency models
Distributed systems -> Messaging/Streaming -> EDA -> CQRS/ES -> Sagas
Virtualization -> Cloud -> Containers -> Kubernetes -> Mesh / Serverless -> Platform engineering
All layers -> Security, Observability, Reliability (cross-cutting)
```

13.5 C. Problem → solution → new problem

Problem	Solution	New problem
CPU idle on I/O	Multiprogramming/threads	Races, deadlocks
Disk slow	Caches, WAL, LSM	Invalidation, compaction
Shared data across users	Client-server DB	Server bottleneck
Fat client deployment	Web/3-tier	Server load
Single DB limit	Replicas, cache, shard	Lag, staleness, cross-shard ops
Single node failure	Replication/failover	Split-brain, consistency
Coordination across nodes	Consensus	Latency, complexity
Monolith coupling	SOA/microservices	Distributed complexity
Cross-service atomicity	Sagas/outbox	Eventual consistency
Slow server provisioning	Cloud/IaaS	Cost sprawl
Env drift	Containers	Orchestration
Cross-cutting network logic	Mesh/sidecar	Overhead
Idle capacity cost	Serverless	Cold start, lock-in
Global latency	CDN/edge/multi-region	Consistency, cost
Batch too slow	Streaming	Late data, state
Keyword search limits	Vector search/RAG	Recall, hallucination controls

13.6 D. Architecture comparison matrix

	Monolith	Modular mono	SOA	Microservices	Serverless	Event-driven	Actor	P2P
Deploy simplicity	★★★	★★★	★	★	★★	★★	★★	★
Independent scaling	★	★	★★	★★★	★★★	★★★	★★★	★★★

	Monolith	Modular mono	SOA	Microservices	Serverless	Event-driven	Actor	P2P
Consistency ease	★★★	★★★	★★	★	★	★	★★	★
Ops burden (higher worse)	low	low	high	high	low–med	med–high	med	high
Team autonomy	★	★★	★★	★★★	★★★	★★★	★★	n/a
Debuggability	★★★	★★★	★★	★	★	★	★	★

13.7 E. Concept checklist (tick when you can explain, trade-offs, and failure modes)

Foundations ☐ OS/concurrency ☐ Networking ☐ App architectures ☐ APIs ☐ Databases/indexes ☐ Transactions/isolation ☐ Replication/sharding ☐ Consistency/CAP/PACELC ☐ Consensus ☐ Caching ☐ Messaging/streams ☐ Reliability/SLOs ☐ Security/AuthN/Z ☐ Observability ☐ CI/CD/IaC ☐ Containers/K8s ☐ Serverless/edge ☐ Data engineering ☐ ML/RAG systems ☐ Cost/FinOps ☐ Governance/privacy ☐ DR.

13.8 F. Commonly omitted by tutorials

Storage-engine internals (compaction, write amplification) · fsync/lying disks · clock skew & fencing tokens · idempotency design · retry budgets · tail latency math · schema evolution/expand-contract · backup restore testing · connection-pool sizing · cache invalidation races · TLS/cert lifecycle · cardinality costs · operational/human factors · multi-tenancy isolation · GDPR deletion in logs/backups · egress costs · capacity N+1 under AZ loss · training/serving skew · prompt-injection in RAG.

13.9 G. Glossary (selected; full terms are defined in each part's tables)

ACID atomic, consistent, isolated, durable txns · **Backpressure** consumer slows producer · **Bloom filter** probabilistic set · **CAP** consistency/availability/partition tolerance · **CDC** change data capture · **CRDT** conflict-free replicated data type · **DLQ** dead-letter queue · **ISR** in-sync replicas · **LSM** log-structured merge tree · **MVCC** multi-version concurrency control · **Quorum** majority agreement · **RPO/RTO** data-loss/time-to-recover targets · **Saga** compensating transaction chain · **SLI/SLO/SLA** indicator/objective/agreement · **Idempotency** repeat-safe operation · **Watermark** event-time progress marker · **mTLS** mutual TLS · **ANN** approximate nearest neighbor · **HNSW** graph ANN index.

13.10 H–I. Learning path & milestones

1. **Beginner:** OS/CPU/memory, networking (DNS/HTTP/TCP), SQL, REST, one language deeply, Git, basic cloud. *Milestone:* build 3-tier CRUD app with auth and deploy it.
2. **Intermediate:** indexing, transactions, caching, queues, LB, Docker, CI/CD, observability basics. *Milestone:* scale an app to replicas + cache + async worker with dashboards.
3. **Advanced:** replication/sharding, consensus, consistency models, Kafka, K8s, sagas, SLOs, security architecture. *Milestone:* design/operate multi-service system with failure drills.
4. **Expert:** multi-region, storage-engine design, capacity/cost modeling, org/architecture strategy, AI platform design. *Milestone:* lead a migration (strangler) with measurable SLOs and rollback.

13.11 J. Practice questions

1. Why does adding a read replica not fix a write-bound DB? 2. When can a cache make correctness worse? 3. Compute availability of 3 serial 99.95% services. 4. Why is exactly-once delivery impossible

end-to-end? 5. Design idempotent payment API. 6. Explain write skew and how snapshot isolation allows it. 7. Pick a partition key for chat and justify. 8. How does Raft handle a partitioned leader? 9. Why prefer p99 over average? 10. Compare 2PC vs saga for order checkout.

13.12 K. Design exercises

URL shortener · news feed · chat · Dropbox · rate limiter · payment ledger · ticket booking · Kafka-like log · distributed lock service · web crawler · metrics system · RAG over enterprise docs · multi-region active-active shopping cart.

13.13 L. Implementation projects

1. Key-value store with WAL + LSM + bloom filter. 2. Raft implementation (leader election + log replication). 3. Consistent-hash cache cluster. 4. Token-bucket distributed rate limiter. 5. Mini message broker with consumer groups and offsets. 6. Saga orchestrator with outbox. 7. Load balancer with health checks. 8. Tiny search engine (inverted index + BM25). 9. Observability stack (OpenTelemetry → Prometheus/Grafana/Jaeger). 10. RAG service with hybrid retrieval and evals. *(Java options in this repo: extend* `threads/`, `datastructures/custom/`, `designpatterns/`.*)*

13.14 M. Final completeness audit

Prompt category	Covered in	Status
Foundations/hardware/OS/concurrency/filesystems	Part 1	✓
Networking	Part 2	✓
App architectures, APIs	Part 3	✓
Databases, transactions, consistency, recovery, governance	Part 4	✓
Scalability, caching	Part 5	✓
Distributed systems, messaging, streaming	Part 6	✓
Reliability, security, observability	Part 7	✓
DevOps, cloud, containers, serverless, edge, FinOps	Part 8	✓
Design patterns (GoF + enterprise + distributed)	Part 9	✓
45 system types	Part 10	✓
Trade-offs, confused concepts, numbers	Part 11	✓
Decision trees, failed ideas, failure cases	Part 12	✓
Data/AI/emerging, evolution map, learning path	Part 13	✓

Honest limitation: the full 22-heading template is applied in depth to pivotal ★ concepts (microservices, Kafka-style logs, etc.); remaining concepts use condensed table rows covering definition, pressure, trade-offs, failure modes and successors. Any concept can be expanded to the full template on request (ask: "expand using the 22-section template").

Part 14 – Deep Dive A: Foundations and Networking (with examples and diagrams)

14.1 Why the memory hierarchy drives every design

Idea: fast storage is small and expensive; big storage is slow. Systems keep hot data close.

```
 Speed/Cost  ^                          Capacity
   fastest   |  Registers      (bytes)
             |  L1/L2/L3       (KB-MB)
             |  RAM            (GB)
             |  SSD            (TB)
             |  HDD / Object   (PB)
   slowest   |  Network/Remote (unbounded)
```

Worked example – why Redis in front of Postgres helps: a Postgres indexed read costs ~1 ms (buffer-pool hit) up to ~10 ms (disk). A Redis read over the LAN costs ~0.3 ms. At 95% hit ratio: `0.95×0.3 + 0.05×10 = 0.785 ms` vs 10 ms uncached – a 12× gain, *and* the DB sees 20× fewer queries.

Sequential vs random I/O example: an HDD does ~100 random IOPS (~0.4 MB/s with 4 KB reads) but ~150 MB/s sequential. This is why Kafka appends to a log, databases write a WAL first, and LSM trees batch writes into sorted files.

```
Random write (B-tree, in place):    Sequential write (LSM / WAL):
 page 17 -> disk seek                 [rec1][rec2][rec3][rec4] -> append only
 page 902 -> disk seek                 no seeks, high throughput
```

14.2 Processes, threads, locks – a runnable Java example

```
// Race condition and fix
class Counter {
    private int unsafe = 0;                       // not thread-safe
    private final java.util.concurrent.atomic.AtomicInteger safe =
        new java.util.concurrent.atomic.AtomicInteger();   // CAS-based

    void incUnsafe() { unsafe++; }                // read-modify-write: 3 steps, interleavable
    void incSafe()   { safe.incrementAndGet(); }  // atomic compare-and-swap loop
}
// Two threads each doing 1,000,000 incUnsafe() typically end < 2,000,000.
```

```
Thread A: read 5 ----------- write 6
Thread B:      read 5 -- write 6          <- lost update (should be 7)
```

Deadlock example: T1 locks A then B; T2 locks B then A.

```
T1: lock(A) ---- wait(B)
T2: lock(B) ---- wait(A)      => cycle => deadlock
Fix: always acquire locks in global order (A then B), or use tryLock(timeout).
```

Context-switch cost → design: 10,000 threads blocked on I/O waste memory (~1 MB stack each) and CPU switching. Response: event loops (Netty/Node), async I/O (epoll), Java virtual threads, or actors.

14.3 Filesystem journaling example

```
Without journal: write data block -> CRASH -> metadata not updated -> corrupted file
With journal:    1. write "intent" to journal  2. commit marker
                 3. apply to real location       4. clear journal
On reboot: replay committed entries, discard incomplete ones.
```

The same *log-first* idea appears in database WAL, Raft logs, and Kafka.

14.4 Serialization comparison (same record `{id:1, name:"Ann"}`)

Format	Size	Human-readable	Schema evolution	Typical use
JSON	~24 B	Yes	Loose, fragile	Public APIs
Protobuf	~8 B	No	Field numbers; add optional fields safely	gRPC/internal
Avro	~7 B	No	Writer/reader schema resolution	Kafka/data lakes
Rule: never reuse/renumber Protobuf field numbers; add fields, never repurpose.				

14.5 What happens when you type a URL (end-to-end diagram)

```
Browser
  | 1. DNS lookup: browser cache -> OS cache -> resolver -> root -> TLD -> authoritative
  v
IP address (GeoDNS may return nearest region / anycast IP)
  | 2. TCP 3-way handshake (SYN, SYN-ACK, ACK)  [QUIC: 1 RTT incl. TLS]
  | 3. TLS 1.3 handshake (certificate check, key exchange)
  v
CDN edge  --hit--> return cached response
  | miss
  v
L4 Load Balancer --> L7 Load Balancer / API Gateway (authN, rate limit, routing)
  v
App instance --> Redis cache --miss--> Database
  v
Response: gzip/brotli, Cache-Control, ETag
```

Latency budget example (user in Mumbai, origin in Virginia): DNS 20 ms + TCP 180 ms + TLS 180 ms + request 180 ms ≈ 560 ms. Put a CDN edge in Mumbai (RTT 10 ms): ≈ 40 ms. That is why CDNs, keep-alive, TLS 1.3 and HTTP/3 exist.

14.6 TCP essentials with examples

- **Handshake:** `SYN → SYN-ACK → ACK` (1.5 RTT before data).
- **Flow control:** receiver advertises window; if window = 0 the sender pauses (cousin of application backpressure).
- **Congestion control:** slow start doubles the window each RTT until loss, then backs off (AIMD). Short flows never leave slow start → keep connections alive and pooled.
- **Head-of-line blocking:** one lost packet stalls all later data on that TCP stream, even for unrelated HTTP/2 requests → QUIC gives independent streams.

```
HTTP/1.1 : [req1][resp1] [req2][resp2] ...             (serial per connection; 6 parallel conns)
HTTP/2   : req1,req2,req3 interleaved on 1 TCP conn     (TCP HOL on loss)
HTTP/3   : req1,req2,req3 as independent QUIC streams   (loss affects only one stream)
```

14.7 DNS, NAT, subnetting examples

- **DNS TTL failover:** `api.shop.com A 10.0.0.5 TTL=300`. Region dies → update record → clients may still use old IP up to 5 min. Use low TTL + health-checked routing; remember resolvers/JVM may cache longer (Java `networkaddress.cache.ttl`).
- **CIDR:** `10.0.0.0/16` = 65,536 addresses; split into `/24` subnets (256 each): `10.0.1.0/24` public, `10.0.2.0/24` app, `10.0.3.0/24` DB.

```
VPC 10.0.0.0/16
 ├─ public subnet 10.0.1.0/24  (LB, NAT gateway)  -> Internet Gateway
 ├─ private subnet 10.0.2.0/24 (app servers)      -> egress via NAT
 └─ private subnet 10.0.3.0/24 (databases)        -> no internet route
```

- **NAT:** many private hosts share one public IP via port mapping; limits ~64K ports per destination → "port exhaustion" for chatty services (use connection pooling).

14.8 Load balancing examples

Algorithm	Example behaviour	Best for
Round robin	1,2,3,1,2,3	Homogeneous stateless
Weighted RR	3:1 to big:small box	Mixed hardware
Least connections	Send to fewest active	Long/variable requests
Power of two choices	Pick 2 random, choose less loaded	Large fleets, avoids herding
Consistent hash	`hash(userId)` → same node	Caches, stateful routing

```
L4: Client --TCP--> LB (NAT/DSR) --> Server        (fast, no HTTP awareness)
L7: Client --HTTP--> LB --parse path/headers--> /api -> svcA, /img -> svcB, retries, TLS termination
```

14.9 Proxies, CDN, anycast

```
Forward proxy:  Clients ──> [Proxy] ──> Internet      (hides clients, filters egress)
Reverse proxy:  Internet ──> [Proxy] ──> Servers      (hides servers, TLS, cache)
Anycast:        same IP 1.2.3.4 announced from Tokyo, Paris, NYC -> BGP routes user to nearest
```

CDN invalidation example: deploy `app.js` with a content hash in the name (`app.3fa9c1.js`) and `Cache-Control: max-age=31536000, immutable`; HTML references the new name. No purge needed – *versioned URLs beat invalidation.*

14.10 Security in the network

```
Internet -> WAF/DDoS shield -> [DMZ: LB] -> [App subnet] -> [DB subnet]
             rate limit        TLS end      SG: only LB->App:8080   SG: only App->DB:5432
```

Zero trust adds: every hop authenticated (mTLS), not just the perimeter.

Part 15 – Deep Dive B: Architectures and APIs (with examples and diagrams)

15.1 Evolution in pictures

```
[Mainframe]           [2-tier]              [3-tier]               [Microservices]
 terminals --> host    fat client --> DB     browser -> app -> DB    gateway -> many svcs -> own DBs
```

15.2 Monolith → modular monolith → microservices (same shop app)

Monolith

```
+-----------------------------------------+
|  Web | Catalog | Cart | Orders | Payments  |   one process, one DB, ACID
+-----------------------+-----------------+
                        v
                    [ One DB ]
```

`orderService.place()` calls `paymentService.charge()` **in-process** inside one DB transaction. Easy, atomic. Pain: one bug can take everything down; to scale Catalog you scale all.

Modular monolith – same deployable but modules only talk through public interfaces:

```
// module boundary: other modules may only use this interface
public interface PaymentApi { PaymentResult charge(OrderId id, Money amount); }
// internal classes are package-private; enforce with ArchUnit / Java modules
```

Microservices – `charge()` becomes a network call:

```
Orders --HTTP/gRPC--> Payments    (timeout? retry? duplicate charge? => idempotency key)
Orders --OrderPlaced event--> Broker --> Inventory, Email
```

Migration (Strangler Fig):

```
Phase 1: Client -> Facade -> [Monolith]
Phase 2: Client -> Facade -> /payments -> [New Payments svc]  ; rest -> Monolith
Phase 3: repeat until monolith is empty, then retire.
```

15.3 Layered, Hexagonal, Clean – in code shape

```
Layered:            Hexagonal (Ports & Adapters):
 Controller            [REST adapter] ->  (port) ->  DOMAIN  <- (port) <- [JPA adapter]
 Service                                              core    <- (port) <- [Kafka adapter]
 Repository -> DB      Domain depends on nothing; adapters depend on domain.
```

```
interface OrderRepository { void save(Order o); }          // port (in domain)
class JpaOrderRepository implements OrderRepository { ... } // adapter (infrastructure)
class PlaceOrder { PlaceOrder(OrderRepository r){...} }     // use-case depends on port only
```

Benefit: test the domain with an in-memory fake. Cost: extra interfaces for trivial CRUD – don't use for a 3-table app.

15.4 MVC / MVP / MVVM

```
MVC : View -> Controller -> Model -> View (observes)
MVP : View <-> Presenter <-> Model       (Presenter updates a passive View)
MVVM: View <==binding==> ViewModel <-> Model
```

Example: Spring MVC controller returns a model + view name; Angular/WPF use two-way binding (MVVM).

15.5 REST vs RPC vs GraphQL – same query three ways

Need: user 42 with their last 3 orders.

```
REST     : GET /users/42 ; GET /users/42/orders?limit=3        (2 round trips, maybe over-fetch)
gRPC     : rpc GetUserWithOrders(GetReq{id:42,limit:3})         (1 call, typed, binary)
GraphQL  : query { user(id:42){ name orders(first:3){ id total } } }   (1 call, client picks fields)
```

Concern	REST	gRPC	GraphQL
Caching	HTTP caching built-in	Manual	Hard (single POST) – persisted queries
Perf	Medium	High	Variable (N+1, deep queries)
Browser	Native	grpc-web	Native
Risk	Chatty	Debugging binary	Query-cost attacks – limit depth/complexity

REST design example

```
POST   /orders            201 Created + Location: /orders/9      (Idempotency-Key header)
GET    /orders/9          200 ; ETag for caching
PATCH  /orders/9          partial update ; If-Match for optimistic concurrency
DELETE /orders/9          204
GET    /orders?cursor=abc&limit=50   (cursor pagination beats offset at scale)
```

Versioning: URL (`/v2/`) or header; never break existing fields – only add.

15.6 Real-time delivery options

```
Polling       : client asks every N s            (simple, wasteful, delayed)
Long polling  : server holds request until event (works anywhere, many open conns)
SSE           : one-way server->client stream over HTTP, auto-reconnect (live feed, LLM tokens)
WebSocket     : full duplex TCP channel           (chat, games, trading)
Webhook       : server->server HTTP callback      (payment succeeded)
```

Webhook receiver rules: verify HMAC signature, respond 2xx fast, process async, be idempotent (provider retries).

WebSocket scaling diagram

```
Clients <--ws--> [WS gateway 1] \
Clients <--ws--> [WS gateway 2]  >-- Pub/Sub (Redis/Kafka) -- [Chat service]
Clients <--ws--> [WS gateway 3] /      user->gateway map in Redis
```

To send to user U: look up gateway holding U → publish to it. Reconnect storms after deploy → jittered reconnect.

15.7 Event-driven, CQRS, Event Sourcing – example

Bank account:

```
Commands -> [Write model] --append--> Event Store: Deposited(100), Withdrawn(30), Deposited(5)
                                              |
                                      projector (async)
                                              v
                                [Read model: balance=75, statement view]  <- Queries
```

- State = replay of events (`100 - 30 + 5 = 75`). Snapshots every N events speed up rebuild.
- **Eventual consistency:** after a command, the read model lags by ms–s; UI can show the command result optimistically.
- **Schema evolution:** events are forever – version them (`Deposited_v2`) and write *upcasters*.
- **GDPR deletion:** encrypt personal data per-user and delete the key (crypto-shredding).

15.8 Actor model example

```
[Actor: Cart-42]  mailbox: AddItem, Checkout, ...  (processes one message at a time, no locks)
       | supervises
[Actor: Payment-42]  crash -> supervisor restarts it (let-it-crash)
```

Good for per-entity state (chat room, device, game session). Weak spot: cross-actor transactions, unbounded mailboxes (need bounded queues/backpressure).

15.9 Pipes & Filters, Microkernel, Space-based, P2P, Blackboard

```
Pipes&Filters:  cat log | grep ERROR | sort | uniq -c    (Unix) ; ETL stages ; Spring Integration
Microkernel  :  [Core] + plugins (IDE, browsers, OSes)    -> stable core API, extensions load dynamically
Space-based  :  processing units each with in-memory data grid; replication between units; DB
asynchronous behind
P2P          :  peers discover via DHT (Kademlia); no central server (BitTorrent)
Blackboard   :  shared workspace; specialist agents read/write until solution emerges (speech
recognition; modern multi-agent LLMs)
```

15.10 API gateway vs BFF vs service mesh

```
 Mobile --> [BFF-mobile] \
 Web    --> [BFF-web]     >-- API Gateway (authN, quotas, routing) --> services
 Partner-> [Public API]  /                                              | east-west traffic
                                                                [sidecar proxies = service mesh: mTLS,
retries, metrics]
```

Gateway = north-south; mesh = east-west. Don't put business logic in the gateway.

15.11 Serverless example & caveats

```
Upload image -> S3 event -> Lambda(resize) -> S3 thumbnails -> DynamoDB metadata
```

Cold start (100 ms–seconds), 15-minute limit, concurrency limits, per-invocation billing. Cost crossover: steady 24×7 high-QPS load is usually cheaper on containers/VMs.

Part 16 – Deep Dive C: Databases and Transactions (with examples and diagrams)

16.1 Indexes and storage engines

Without an index `SELECT * FROM orders WHERE user_id=42` scans every row (O(n)). With a B+ tree index on `user_id` it is O(log n).

```
B+ tree (fan-out ~100-500):            Root:      [ 30 | 60 ]
 3-4 levels hold billions of rows      Internal:  [10|20] [40|50] [70|80]
 leaves are linked -> range scans      Leaves: [..]->[..]->[..]->[..]   (linked list; hold row
pointers/rows)
```

- **Composite index** `(user_id, created_at)` serves `WHERE user_id=? ORDER BY created_at`; leftmost-prefix rule: it does *not* help `WHERE created_at=?` alone.
- **Covering index** includes all selected columns → no table lookup.
- **Cost:** each index slows writes and uses space. Index selectivity matters (an index on a boolean is rarely useful).
- Check plans: `EXPLAIN ANALYZE SELECT ...`.

LSM tree write path

```
write -> WAL (durable) -> Memtable (sorted, in RAM)
          memtable full -> flush -> SSTable L0 -> compaction merges into L1, L2 ...
read  -> memtable -> bloom filter -> SSTables newest->oldest
```

Trade-off: writes sequential and fast; reads touch several files (bloom filters help); compaction causes write amplification and latency spikes.

	B-tree	LSM
Writes	Random, in-place	Sequential, batched
Reads	1 path	Multiple levels
Space amp	Low-ish	Higher until compaction
Fit	Read-heavy OLTP	Write-heavy ingestion

Bloom filter example: m bits, k hashes. Query "key X?" → if any bit is 0 → *definitely absent* (skip disk); if all 1 → *maybe present*. 10 bits/key, 7 hashes ≈ 1% false positives.

16.2 Modeling example – normalization vs denormalization

Normalized:

```
users(id, name)   orders(id, user_id FK, total)   order_items(order_id, product_id, qty, price)
```

Report "orders with user name" = join. Denormalized order document in a document DB:

```
{ "orderId": 9, "user": {"id":42,"name":"Ann"}, "items":[{"sku":"A1","qty":2,"price":10}], "total":20 }
```

Fast single read, but if Ann changes name, old orders keep the old copy (often *desired* for invoices – a snapshot). Rule: normalize for integrity, denormalize deliberately for read paths.

Primary key choice: auto-increment (compact, sequential – hot right-edge in sharded DBs, guessable) vs UUIDv4 (random → index fragmentation) vs UUIDv7/Snowflake (time-ordered, unique without coordination).

16.3 Transactions and isolation – concrete anomalies

Bank: A=100, B=100.

```
Dirty read:           T1 updates A=50 (uncommitted); T2 reads A=50; T1 rolls back -> T2 saw phantom value
Non-repeatable read:  T1 reads A=100; T2 commits A=50; T1 reads A=50
Phantom:              T1 counts rows where x>10 =5; T2 inserts a row; T1 counts again =6
Lost update:          T1 and T2 both read 100, both write back +10 -> 110 instead of 120
Write skew:           Doctors on call rule "at least 1": T1 and T2 each see 2 on call, each takes self
off -> 0 on call
                      (both pass under snapshot isolation; needs Serializable or explicit locking)
```

Isolation level	Dirty	Non-rep.	Phantom	Write skew
Read Uncommitted	possible	possible	possible	possible
Read Committed (Postgres default)	no	possible	possible	possible
Repeatable Read / Snapshot	no	no	mostly no*	possible
Serializable	no	no	no	no
(*implementation-specific)				

Fixing lost update

```
-- pessimistic: lock row
BEGIN; SELECT balance FROM acct WHERE id=1 FOR UPDATE; UPDATE acct SET balance=balance+10 WHERE id=1;
COMMIT;
-- atomic update (best): UPDATE acct SET balance = balance + 10 WHERE id=1;
-- optimistic: UPDATE ... SET balance=?, version=version+1 WHERE id=1 AND version=?  (retry if 0 rows)
```

MVCC picture

```
row key=1: [v1 val=100 xmin=5] -> [v2 val=90 xmin=9]
Reader with snapshot @7 sees v1 ; reader @10 sees v2 ; writers don't block readers
Vacuum/GC removes versions no snapshot can see.
```

WAL/recovery:

```
UPDATE -> write log record (LSN 120: page 7, old=100, new=90) -> fsync log -> COMMIT ack
Page flushed later. Crash: ANALYSIS (what was active) -> REDO (replay committed) -> UNDO (roll back
losers)
Checkpoint bounds how much log must be replayed.
```

16.4 Replication in practice

```
Leader-follower:
 App --writes--> [Leader] --WAL stream--> [Follower1]  <-- reads (may be stale)
                                    \---> [Follower2]
```

- **Replica lag problem:** user posts a comment (to leader) then refreshes (reads follower) → comment missing. Fixes: read-your-writes (route that user to leader for N seconds, or session token with LSN), monotonic reads (stick user to one replica).
- **Failover risk:** async replica promoted; last few transactions lost (RPO>0). Old leader returns → must be fenced/demoted to avoid split-brain.
- **Multi-leader:** two regions accept writes → conflict e.g. both edit title. Resolve: last-write-wins (loses data), merge via CRDT, or app-level.
- **Leaderless quorum:** N=3, W=2, R=2 ⇒ R+W>N so reads overlap latest write.

```
write x=5 -> nodes A,B ack (W=2), C missed
read -> asks A,C (R=2): A has x=5, C has old -> return newest (by version/vector clock) + read repair C
```

16.5 Sharding example

Users 1 B rows, 1 KB each = 1 TB; 20 K writes/s too many for one node → shard.

```
shard = hash(user_id) % N   (simple, but changing N moves almost all keys)
consistent hashing ring:    hash ring with vnodes; add node -> only ~1/N keys move

 ring:  0 ---A---B---C---A---B---C--- 2^32   (vnodes spread load; key goes to next node clockwise)
```

- Cross-shard query ("all orders over $100") = scatter-gather. Cross-shard transaction = saga/2PC. Choose shard key to keep related data together (tenant_id, user_id).
- **Hot shard example:** celebrity user → key salting (`userId#0..9`) spreads writes; reads aggregate.
- **Range sharding hotspot:** timestamp key means all current writes hit last range; prefix with hash bucket.

16.6 CAP & PACELC with examples

```
Partition occurs between DC1 and DC2:
 CP choice: minority side rejects writes (etcd, Spanner, ZooKeeper)  -> consistent, partly unavailable
 AP choice: both accept writes, reconcile later (Cassandra, Dynamo)  -> available, possible conflicts
```

PACELC: even with no partition, strong consistency costs latency (must wait for replicas); Dynamo-style systems choose low latency (EL), Spanner chooses consistency (EC).

16.7 Distributed transactions – 2PC vs Saga example (order checkout)

```
2PC:  Coordinator -> prepare -> [Orders DB][Payments DB][Inventory DB] -> all vote yes -> commit
      Locks held during the protocol; coordinator crash = participants blocked.
Saga (orchestrated):
  1 Create order (PENDING) -> 2 Reserve stock -> 3 Charge card -> 4 Confirm order
  failure at 3: compensate: Release stock -> Cancel order
```

Compensations must be *idempotent* and business-meaningful (refund ≠ undo). Use the **outbox** so state change and event are atomic:

```
BEGIN; INSERT orders ...; INSERT outbox(event=OrderPlaced); COMMIT;
Relay/Debezium reads outbox -> publishes to Kafka (at-least-once) -> consumers dedupe via inbox table.
```

16.8 Backup / DR example

```
Sunday: full backup  | Mon-Sat: incremental | Continuous: WAL archive -> PITR to any second
Replica in another region (DR). Quarterly: restore into scratch env, run checksums, time it (= real RTO).
```

`DROP TABLE users;` replicates instantly to all replicas – only backups/PITR save you.

16.9 Choosing a store – worked scenarios

Scenario	Pick	Reason
Banking ledger	PostgreSQL / Spanner	Serializable, audit
Shopping cart	DynamoDB/Redis	Key access, TTL, high availability
Product search	Elasticsearch fed by CDC	Full-text + facets
IoT metrics 1 M points/s	TimescaleDB/ClickHouse/Kafka+TSDB	Append + compression
Social graph "friends of friends"	Graph DB / adjacency in KV	Traversals
Semantic doc search	pgvector → dedicated vector DB at scale	ANN

Part 17 – Deep Dive D: Scalability, Caching, Distributed Systems, Messaging

17.1 Scaling journey of one app (stage by stage)

```
Stage 1: [Client] -> [App+DB on one box]                                  ~100 users
Stage 2: [Client] -> [App] -> [DB]  (separate boxes)                        ~1K
Stage 3: [Client] -> [LB] -> [App x N, stateless] -> [DB primary + replicas]  ~10K-100K
Stage 4: + [Redis cache] + [CDN] + [Queue + workers] for async jobs          ~1M
Stage 5: + sharded DB / NoSQL, search index, service split, multi-AZ        ~10M+
Stage 6: + multi-region, edge, cell architecture                             global
```

Rule: *measure, find the bottleneck, apply the cheapest fix*; every step adds failure modes.

Making an app stateless – before/after

```
Before: session in server memory -> LB needs sticky sessions; deploy loses sessions
After : session id in cookie -> Redis (or signed JWT) -> any instance can serve
```

17.2 Capacity example

Target 5,000 req/s peak; each request uses 20 ms CPU; 4 cores/instance. `per-instance = 4 / 0.020 = 200 req/s` → need 25 instances at 100% → at 60% target utilization ≈ 42, plus N+1 (AZ loss with 3 AZs: ×1.5) ≈ 63. **DB connections:** 63 instances × pool 20 = 1,260 connections > Postgres comfortable ~200–500 → use PgBouncer or smaller pools (pool size ≈ QPS × avg query seconds).

17.3 Rate limiting algorithms

```
Token bucket (capacity 10, refill 5/s): each request takes a token; bursts up to 10; avg 5/s.
Leaky bucket: queue drains at fixed rate (smooths output).
Fixed window: count per minute (boundary burst: 2x at the edge).
Sliding window log/counter: accurate, more memory.
```

```
// simple token bucket
class TokenBucket {
  private final long cap, refillPerSec; private double tokens; private long last = System.nanoTime();
  TokenBucket(long cap,long r){this.cap=cap;this.refillPerSec=r;this.tokens=cap;}
  synchronized boolean tryAcquire(){
    long now=System.nanoTime(); tokens=Math.min(cap,tokens+(now-last)/1e9*refillPerSec); last=now;
    if(tokens>=1){tokens--;return true;} return false; }
}
```

Distributed: Redis Lua script / `INCR` + `EXPIRE`; per API key and per IP; return `429` + `Retry-After`.

17.4 Tail latency, hedging, fan-out

```
Aggregator --> 100 shards in parallel; response time = slowest shard
p99 per shard 100 ms  ->  P(at least one slow among 100) = 1 - 0.99^100 = 63%
Hedge: if no reply by p95, send same (idempotent) request to a replica; take first -> cuts tail at ~5%
extra load.
```

17.5 Caching – patterns with code and races

```
// cache-aside
Product get(long id){
  Product p = cache.get(id);
  if(p!=null) return p;
  p = db.find(id);                 // miss
  if(p==null) cache.put(id, NULL_MARKER, 60);   // negative caching (penetration)
  else cache.put(id, p, 300 + jitter());        // TTL + jitter (avalanche)
  return p;
}
void update(Product p){ db.save(p); cache.delete(p.id); }   // invalidate after write
```

Race: T1 reads old from DB → T2 updates DB and deletes cache → T1 writes old value into cache (stale until TTL). Mitigate: short TTL, versioned values (compare-and-set), delayed second delete, or don't cache critical data.

Stampede / single-flight

```
1000 requests miss simultaneously -> only first loads from DB; others wait for same Future (single-
flight/lock) or get stale value (stale-while-revalidate)
```

Write strategies

```
write-through: App -> Cache -> DB (sync)        safe, slower writes
write-back   : App -> Cache ... async flush DB   fast, data loss if cache dies
```

Eviction example (capacity 3, access A B C A D): LRU evicts B (least recently used) when D arrives; LFU evicts the least frequently used; FIFO evicts A.

Multi-level

```
Request -> L1 Caffeine (in-process, µs) -> L2 Redis (ms) -> DB
Invalidation: publish "key X changed" on pub/sub so L1 instances drop X.
```

17.6 Distributed-systems failure handling – code and timelines

Retry with exponential backoff + full jitter

```
for (int attempt=0; attempt<5; attempt++) {
  try { return call(); }
  catch (TransientException e) {
    long cap = Math.min(10_000, 100L << attempt);        // 100ms,200ms,400ms... capped
    Thread.sleep(ThreadLocalRandom.current().nextLong(cap));   // full jitter
  }
}
```

Without jitter all clients retry at the same instants → synchronized load spikes.

Circuit breaker states

```
CLOSED --failures>threshold--> OPEN --after cool-down--> HALF-OPEN --probe ok--> CLOSED
                               ^                          | probe fails
                               +--------------------------+
OPEN: fail fast / fallback (cached data) -> protects the dependency and your threads.
```

Retry storm example: 3 layers each retry 3× → one user request becomes 3×3×3 = 27 calls to the bottom service. Fix: retry only at one layer, retry budgets (≤10% extra), deadlines propagated downward.

Safe distributed lock needs fencing

```
Client A gets lock (token 33) -> long GC pause -> lease expires
Client B gets lock (token 34) -> writes to storage
Client A wakes, writes with token 33 -> storage rejects (33 < 34)
```

Leader election with Raft (simplified)

```
Follower --no heartbeat timeout--> Candidate (term+1) --majority votes--> Leader
Leader appends entry -> replicates to majority -> commit -> apply -> reply
Minority-partition leader cannot commit (no majority) -> steps down.
```

Clocks: Lamport: on send `t++`, on receive `t=max(local,msg)+1`. Vector clock `[A:2,B:1]` vs `[A:1,B:2]` = concurrent (conflict); `[A:2,B:1]` vs `[A:1,B:1]` = first is newer.

Gossip: each node every second tells 1–3 random peers its state → information reaches all N nodes in ~O(log N) rounds; failure detected via missing heartbeats (SWIM).

17.7 Messaging – examples

Kafka partitioning & ordering

```
Topic orders (3 partitions), key = orderId
 P0: o1:created, o1:paid, o4:created
 P1: o2:created, o2:cancelled
 Order preserved per key (same partition), not across keys.
Consumer group G1 (3 consumers) -> each owns one partition; 4th consumer idle. Lag = log-end offset –
committed offset.
```

Delivery semantics in code (at-least-once consumer)

```
for (ConsumerRecord r : consumer.poll(...)) {
   if (inbox.alreadyProcessed(r.key(), r.offset())) continue;   // idempotency
   process(r);                                                   // side effect + inbox insert in ONE txn
}
consumer.commitSync();   // commit after processing -> crash causes reprocessing, not loss
```

RabbitMQ-style routing: `producer -> exchange(topic) --"order.*.eu"--> queue A`, `"order.#" --> queue B`; ack/nack, prefetch limits for backpressure, DLX for dead letters.

Retry + DLQ flow

```
main queue --fail--> retry-5s --fail--> retry-1m --fail--> DLQ (alert, inspect, redrive)
```

Windowing example (event time): clicks per minute.

```
Tumbling: [00:00-01:00) [01:00-02:00)       Sliding(1m size, 30s slide): overlapping windows
Session: gap 30s of inactivity closes the window
Watermark = max event time – 5 min allowed lateness; events later than that go to side output.
```

Lambda vs Kappa

```
Lambda: source -> batch layer (accurate, hourly) + speed layer (approx, real-time) -> serving merges
Kappa : source -> replayable log -> one stream job (reprocess by replaying with new code)
```

ETL vs ELT: ETL: extract → transform in pipeline → load clean data. ELT: load raw to warehouse → transform with SQL (dbt) – cheap scalable compute, keeps raw data.

17.8 Durable execution example (Temporal-style)

```
workflow checkout(order):
   reserveStock(); chargeCard(); sendEmail()      // each step is a retried "activity"
State/history persisted; if worker crashes it replays history and resumes at the next step.
Rule: workflow code must be deterministic (no random/time directly).
```

Part 18 – Deep Dive E: Reliability, Security, Observability, Deployment

18.1 Availability math – worked examples

```
Serial chain: LB(99.99) -> App(99.95) -> DB(99.95)
  A = 0.9999 × 0.9995 × 0.9995 = 99.89%   (~9.6 h downtime/year)
Redundant DB pair (independent, 99.95% each): 1-(0.0005)^2 = 99.99975%
But failover isn't instant and failures correlate (same AZ/bug/config) -> real figure lower.
```

Error budget: SLO 99.9%/30 days → budget 43.2 min. Burn-rate alert: consuming 2% of budget in 1 hour (14.4× burn) pages someone; slow burn opens a ticket.

DR strategies (cost ↑, RTO/RPO ↓)

```
Backup & restore : RTO hours, RPO hours       cheapest
Pilot light      : core DB replicated, app off; RTO ~tens of minutes
Warm standby     : scaled-down full stack;      RTO minutes
Active-active    : multi-region both serve;     RTO ~0, RPO ~0 (hard: conflicts, cost)
```

Multi-AZ example

```
Region
 ├─ AZ-a: LB node, App x2, DB primary
 ├─ AZ-b: LB node, App x2, DB sync standby   (auto failover ~60 s)
 └─ AZ-c: App x2, quorum witness
Lose an AZ -> remaining capacity must carry 100% (run each AZ ≤ 66%).
```

Chaos experiment template: hypothesis ("checkout stays above 99.5% success if one cache node dies") → inject (kill node) → observe SLI → limit blast radius → fix gap.

18.2 Security – flows and examples

OAuth 2.0 Authorization Code + PKCE (login with Google)

```
User -> App: "Login"
App -> Browser: redirect to IdP /authorize?client_id&code_challenge&scope=openid
User authenticates at IdP, consents
IdP -> Browser -> App: redirect with ?code=abc
App -> IdP /token: code + code_verifier  -> {access_token, id_token (OIDC), refresh_token}
App -> API: Authorization: Bearer <access_token>
```

OAuth = *delegated authorization*; OIDC adds the *identity* layer (`id_token`).

JWT anatomy & pitfalls: `header.payload.signature` (base64url). Validate signature, `exp`, `aud`, `iss`; reject `alg:none`; keep short-lived (5–15 min); revocation requires deny-list or short TTL + refresh rotation. Don't store secrets in payload (it's only encoded).

Password storage: `argon2id(password, unique salt)` – slow on purpose. Never SHA-256 alone; never reversible encryption.

SQL injection example

```
// vulnerable
"SELECT * FROM users WHERE name='" + input + "'"      // input: ' OR '1'='1
// safe
PreparedStatement ps = c.prepareStatement("SELECT * FROM users WHERE name=?"); ps.setString(1,input);
```

XSS: output-encode user content, set `Content-Security-Policy`, `HttpOnly` cookies. **CSRF:** `SameSite=Lax/Strict`, anti-CSRF token. **SSRF:** block requests to `169.254.169.254` / internal ranges, use allow-lists, IMDSv2.

mTLS between services

```
Svc A --(client cert signed by internal CA)--> Svc B verifies cert identity (SPIFFE ID) -> authorize "A
may call /orders"
Short-lived certs auto-rotated by mesh/SPIRE; expiry = outage, so monitor.
```

Envelope encryption

```
Data --AES-256 with DEK--> ciphertext ;  DEK --encrypted by KEK in KMS/HSM--> stored beside data
Rotate KEK cheaply (re-wrap DEKs) without re-encrypting all data.
```

Multi-tenant isolation: every query includes `tenant_id` (enforce via row-level security), per-tenant encryption keys, per-tenant rate limits. **Supply chain:** pin dependency versions + lockfiles, SBOM, signed images, scan CVEs (this repo can use the CVE tooling), minimal base images, non-root containers.

18.3 Observability – examples

Three pillars wired together

```
Request -> trace_id=abc123 (header traceparent)
 logs   : {"ts":..,"level":"ERROR","trace_id":"abc123","svc":"payments","msg":"timeout","order":9}
 metrics: http_request_duration_seconds_bucket{route="/pay",status="500"}
 traces : gateway(5ms) -> orders(40ms) -> payments(900ms!) -> bank-api(880ms)   => culprit found
```

RED dashboard: Rate (req/s), Errors (% 5xx), Duration (p50/p95/p99). **USE:** Utilization, Saturation, Errors for resources (CPU, queue depth). **Percentile example:** 99 requests at 100 ms and 1 at 10 s → average 199 ms hides the 10 s; p99 = 100 ms, p99.9 reveals it. Use histograms, not averages. **Cardinality trap:** metric label `user_id` with 10 M users = 10 M time series → TSDB melts. Put high-cardinality detail in logs/traces, not metric labels. **Alert design:** page on *user-visible symptoms* (SLO burn), not on "CPU 80%". **Incident flow:** detect → declare severity → incident commander → mitigate (rollback first, root-cause later) → communicate → blameless postmortem with action items. **Health checks:** liveness (is process stuck? restart), readiness (can I serve? remove from LB during warm-up or dependency outage), startup probe for slow boots. Don't make liveness depend on the database – you'll restart-loop during a DB outage.

18.4 Deployment strategies – diagrams

```
Rolling   : v1 v1 v1 v1 -> v2 v1 v1 v1 -> v2 v2 v1 v1 -> ... (both versions live; must be compatible)
Blue-green: LB -> BLUE(v1)   ; deploy GREEN(v2), test ; flip LB -> GREEN ; keep BLUE for rollback
Canary    : 1% -> v2 ... watch error/latency SLIs ... 10% -> 50% -> 100% (auto-rollback on regression)
Shadow    : live traffic mirrored to v2, responses discarded (compare outputs; no side effects)
```

Expand-and-contract migration – rename column `name` → `full_name`

```
1 Expand  : ADD full_name (nullable); app writes both
2 Backfill: copy name -> full_name in batches
3 Switch  : app reads full_name
4 Contract: stop writing name; later DROP name
Every step is backward compatible, so rolling deploys and rollbacks are safe.
```

Feature flag: `if (flags.enabled("new-checkout", user)) newFlow(); else oldFlow();` – deploy dark, release gradually, kill switch; delete stale flags.

18.5 Docker & Kubernetes example

```
FROM eclipse-temurin:21-jre
COPY app.jar /app.jar
USER 10001
ENTRYPOINT ["java","-jar","/app.jar"]
```

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: orders }
spec:
  replicas: 3
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  selector: { matchLabels: { app: orders } }
  template:
    metadata: { labels: { app: orders } }
    spec:
      containers:
      - name: orders
        image: registry/orders:1.4.2
        resources: { requests: { cpu: "500m", memory: 512Mi }, limits: { memory: 1Gi } }
        readinessProbe: { httpGet: { path: /ready, port: 8080 } }
        livenessProbe:  { httpGet: { path: /live,  port: 8080 } }
```

```
kubectl apply -> API server -> etcd (desired state)
Controllers reconcile -> Scheduler picks node -> kubelet runs container -> Service/Endpoints route
traffic
HPA scales replicas on CPU/custom metrics; Ingress exposes HTTP; ConfigMap/Secret inject config.
```

Service mesh data/control plane

```
Pod[ app <-> sidecar proxy ]  <--mTLS-->  Pod[ sidecar <-> app ]
            ^ policy/certs/telemetry config
        [ Control plane (Istiod) ]
```

IaC example (Terraform): `resource "aws_db_instance" "main" { engine="postgres" multi_az=true backup_retention_period=7 }` → `plan` shows diff, `apply` converges; keep state remote and locked; detect drift. **GitOps:** merge PR → Argo CD reconciles cluster to Git; rollback = `git revert`.

18.6 Cost examples (FinOps)

- 100 instances × $0.10/h × 730 h ≈ $7,300/month; rightsizing to 60% saves ~$2,900. Spot/preemptible for stateless/batch (−60–90%); reserved/savings plans for steady base load.
- Cross-AZ/region egress ($0.01–0.09/GB) can exceed compute: keep chatty services in same AZ where safe, compress, cache at edge.
- Observability bills grow with log volume and metric cardinality – sample and set retention tiers.

Part 19 – Deep Dive F: Patterns in Code and Full System Designs

19.1 GoF patterns – compact Java examples

```
// Singleton (thread-safe, enum)
enum Config { INSTANCE; String get(String k){ return System.getProperty(k);} }

// Factory method
interface Notifier { void send(String m); }
class NotifierFactory { static Notifier of(String t){ return switch(t){ case "sms"->new Sms(); default->new Email(); }; } }

// Builder
Order o = Order.builder().id(1).item("A1",2).coupon("X").build();

// Strategy (lambda)
interface Pricing { double price(double base); }
Pricing sale = b -> b * 0.9;

// Decorator (Java I/O): new BufferedReader(new InputStreamReader(new FileInputStream(f)))

// Observer
interface Listener { void onEvent(String e); }
class Bus { List<Listener> ls = new CopyOnWriteArrayList<>(); void publish(String e){ ls.forEach(l->l.onEvent(e)); } }

// Proxy (dynamic): Proxy.newProxyInstance(cl, new Class[]{Svc.class}, (p,m,a)->{ log(m); return m.invoke(real,a); });

// Chain of responsibility (servlet filters): auth -> rateLimit -> logging -> handler
// State: OrderState {NEW->PAID->SHIPPED}, each state object permits only valid transitions
// Template method: abstract class Job { final void run(){ init(); doWork(); cleanup(); } abstract void doWork(); }
```

Existing examples in this repo: `designpatterns/creational|structural|behavioural`.

19.2 Distributed patterns – diagrams

Circuit breaker + bulkhead (Resilience4j style)

```
call -> [Bulkhead: max 20 concurrent for payments] -> [CircuitBreaker] -> [TimeLimiter 800ms] -> [Retry 2x backoff] -> service
```

Sidecar / Ambassador / Adapter

```
Pod: [App] <-localhost-> [Sidecar: logging/TLS/retry]   (Sidecar)
     [App] -> [Ambassador proxy] -> remote DB/shards     (Ambassador hides sharding/connection logic)
     [Legacy XML svc] <- [Adapter svc] <- JSON clients   (Adapter service)
```

Competing consumers + claim check

```
Producer -> Queue --> Worker1 / Worker2 / Worker3 (scale by adding workers)
Large video: upload to blob store -> message carries only blob URL (claim check)
```

Scatter-gather: `Search API -> [shard1..shardN] -> merge top-K -> respond (with partial results on timeout)`. **TCC (Try-Confirm-Cancel):** Try = place hold on $50; Confirm = capture hold; Cancel = release hold (holds expire automatically). **Anti-corruption layer:** translate `LegacyCust{cust_nm, sts_cd}` ↔ `Customer{name,status}` so the legacy model never leaks into the new domain. **Idempotency key**

```
POST /payments  Idempotency-Key: k-123
server: if key exists -> return stored response ; else process, store (key, response) in same txn
```

19.3 System design 1 – URL shortener (full)

Requirements: 100 M new URLs/month, 100:1 read:write, redirect p99 < 50 ms, 5-year retention. **Estimates:** writes ≈ 40/s; reads ≈ 4 K/s avg, 20 K/s peak; storage ≈ 3 TB over 5 yrs.

```
Client -> CDN -> LB -> [Redirect svc] -> Redis (code->url) -> [KV store sharded by code]
                     [Create svc]  -> ID generator (ranges of counters per node) -> base62
                     click events -> Kafka -> analytics (OLAP)
```

Key decisions: base62 of unique ID (no collisions) vs hash+collision check; 301 (cacheable, loses analytics) vs 302; expire via TTL; abuse → rate limit + malware blocklist. Failure cases: cache down → KV absorbs with rate limit; ID service down → each node pre-allocates a range.

19.4 System design 2 – Chat

```
Client <--WebSocket--> [Gateway pool] --> [Chat svc] --> [Message store: Cassandra, key=(chat_id,
ts/seq)]
                          |                 |--> [Presence: Redis TTL heartbeat]
                          |                 |--> [Push/Notification svc] for offline users
                          +-- Redis map: user_id -> gateway_id
```

- Ordering: per-conversation sequence number assigned by the partition owner; clients dedupe by message id.
- Group fan-out: small groups fan-out on write; huge channels fan-out on read.
- Delivery states: sent → delivered → read via acks.
- Reconnect storm: jittered backoff; resume from last seq.

19.5 System design 3 – News feed (hybrid fan-out)

```
Post -> [Post svc] -> Kafka -> [Fan-out workers] -> write to followers' feed lists in Redis (normal
users)
Celebrity posts are NOT fanned out; at read: merge precomputed feed + pull recent posts from followed
celebrities
Read: [Feed svc] -> feed cache -> hydrate post/user data -> rank (ML) -> return page (cursor)
```

Trade-off: write amplification (1 post × 1 M followers) vs read latency; hybrid caps both.

19.6 System design 4 – Payment system

```
Client -> API (Idempotency-Key) -> [Payment svc] --state machine: CREATED->AUTHORIZED->CAPTURED-
>SETTLED/REFUNDED
                                    |-> PSP (Stripe-like) ; webhook confirms async result
                                    |-> Ledger DB (double-entry, append-only, serializable)
                                    |-> Outbox -> Kafka -> notifications, accounting
Reconciliation job: compare internal ledger with PSP/bank reports daily; fix mismatches.
```

Timeout-after-send problem: unknown if charge happened → query PSP by idempotency key before retrying. Never use floats for money (use integer minor units).

19.7 System design 5 – Ticket booking (no double booking)

```
Select seat -> HOLD (row seat_id, state=HELD, expires_at=now+5m, SELECT FOR UPDATE / conditional update)
Pay -> state=BOOKED ; timer job/TTL releases expired holds
Flash sale: virtual waiting room (queue) admits N users/sec to protect inventory DB.
```

```
UPDATE seats SET state='HELD', held_by=:u, expires_at=now()+interval '5 min'
WHERE id=:s AND (state='FREE' OR expires_at<now());   -- 1 row updated = success
```

19.8 System design 6 – RAG system

```
Ingest : docs -> parse -> chunk (300-800 tokens, overlap) -> embed -> vector index + BM25 index (+
metadata: ACL, source, date)
Query  : question -> (rewrite) -> hybrid retrieve top-50 -> filter by user ACL -> rerank top-5 -> prompt
+ citations -> LLM -> guardrails -> answer
Eval   : retrieval recall@k, groundedness, answer quality; log traces; user feedback loop
```

Risks: prompt injection from retrieved docs (treat as untrusted), ACL leakage (filter before generation), stale index (incremental re-index via CDC), cost/latency (cache, smaller models for rewrite, stream tokens).

19.9 System design 7 – Distributed rate limiter, KV store, job scheduler (sketches)

```
Rate limiter : Gateway -> Redis cluster (Lua token bucket per key) ; fail-open vs fail-closed policy;
local pre-check
KV store     : client -> coordinator node -> consistent-hash ring -> N=3 replicas (W=2,R=2);
WAL+memtable+SSTable; gossip; hinted handoff
Scheduler    : jobs table (run_at, status) -> workers claim via lease (UPDATE ... WHERE
lease_expires<now() RETURNING) ; fencing token; idempotent jobs
```

19.10 Interview framework (how to present any design)

1. Clarify functional + non-functional requirements (scale, latency, consistency, availability).
2. Estimate (QPS, storage, bandwidth).
3. API + data model.
4. High-level diagram; walk the read and write paths.
5. Deep-dive bottlenecks (cache, sharding, queues, consistency).
6. Failure modes, security, observability, cost.
7. State trade-offs and alternatives explicitly.